Journal of Circuits, Systems, and Computers
© World Scientific Publishing Company

Instruction-Fuel Drift, Boundary-Crossing Overhead, and Memory Safety in WebAssembly Sandboxes: A Multi-Runtime Measurement Study

DEVARSH PATEL
School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology (VIT)
Vellore, Tamil Nadu, 632014, India devarsh.patel2024@vitstudent.ac.in

RANJITHKUMAR S*
School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology (VIT)
Vellore, Tamil Nadu, 632014, India ranjithkumar.s@vit.ac.in

Received (Day Month Year)
Revised (Day Month Year)
Accepted (Day Month Year)

Multi-tenant WebAssembly platforms enforce compute limits by metering instruction fuel. This study investigates the fidelity with which instruction fuel tracks Central Processing Unit (CPU) execution time across twelve runtime configurations on an 8-core 64-bit Advanced RISC Machine (ARM) processor (ARMv8.6-A) within a load-logged environment. Across 50 benchmark kernels and four optimization levels, the CPU duration per fuel unit under Wasmtime's Cranelift backend exhibits a 34- to 38-fold dispersion. Hardware performance counters demonstrate that cycles per native instruction accounts for 84% of the log-variance, with cache, translation lookaside buffer (TLB), and branch events explaining roughly half of the observed spread. An adversarial search raises unit cost to 93.8 ns per fuel unit—260 times the benign maximum—via dependent cache-missing loads. Furthermore, Wasmtime's Winch baseline compiler meters bulk-memory instructions per operation rather than per byte, creating an unbounded accounting loophole that explains every observed discrepancy with Cranelift. Static opcode re-weighting fails under leave-one-kernel-out (LOKO) validation, and hardware-counter models underestimate adversarial execution by up to 28 times. To resolve these vulnerabilities, we evaluate DualGate, a dual-metering mechanism coupling instruction fuel with asynchronous epoch interruption. DualGate confines execution within 6.3% of allocated CPU budgets across benign, serverless, and adversarial workloads on both backends—where fuel alone overshoots by up to 49 times—at an execution overhead of 6.8%. Linear-memory isolation remains intact across 4,800 single and 4,200 compound trials. These findings indicate that fuel constitutes a cooperative yield primitive rather than an equitable CPU quota, governed by compiler backend code generation rather than guest module bytecode.

Keywords: WebAssembly; serverless computing; fuel metering; sandbox isolation; runtime benchmarking; microarchitectural measurement.

*Corresponding author.

1. Introduction

A fundamental requirement of multi-tenant cloud platforms is the execution of untrusted code within lightweight WebAssembly sandboxes to ensure rapid startup latency and strict memory isolation [1,?]. However, memory isolation represents only a partial solution. A central concern in multi-tenant execution is establishing a dependable bound on CPU consumption, which runtimes conventionally enforce by metering instructions via an abstraction termed instruction fuel. The runtime decrements fuel as the guest executes and traps when the allocated budget is exhausted.

Fuel counters track guest instructions, whereas CPU time depends on the native machine code emitted by the compiler. Between the high-level computation and native execution lie two compilation stages: the frontend compiler producing the bytecode module and the backend code generator lowering it to machine instructions. Both stages alter the dynamic work performed, while the backend dictates the concrete cost of each operation. Consequently, a fixed fuel allocation yields substantially disparate CPU durations across different modules, optimization tiers, and runtime engines. Although Jangda et al. evaluated WebAssembly performance relative to native execution across standard benchmarks [2], what is not yet clear is the fidelity with which instruction fuel tracks thread CPU time, and the extent to which this relationship depends on compiler configurations and runtime backends.

This issue has significant practical implications under adversarial workloads. In cloud environments where resource consumption is billed per fuel unit, an untrusted tenant has an incentive to execute computations that maximise CPU consumption per metered unit, as demonstrated by Perez and Livshits in the context of Ethereum Virtual Machine (EVM) gas [3]. Therefore, the present study investigates both the empirical dispersion across benign workloads and the worst-case boundaries an adversarial tenant can construct, using validated hardware performance counters to explain the observed phenomena.

The present study makes six primary contributions:

- Fuel-to-time drift across twelve configurations. For 50 benchmark kernels across four optimization levels, we evaluate the cost of one fuel unit under a load-logged protocol, demonstrating that it spreads by a factor of 34 to 38 under Wasmtime Cranelift in two independent gated runs, and showing that Winch and Cranelift report different fuel for the same module (Section 5.1).
- A microarchitectural account of the drift. Using validated kperf performance monitoring unit (PMU) counters, we establish that cycles per native instruction accounts for 84% of the variance of the log cost per fuel unit, that cache, TLB, and branch events explain about half of it, and that the miss rate per load, not the share of memory operations, is the operative variable (Section 5.2).
- An adversarial worst case and a backend metering gap. A structured search over 39 kernels raises the Cranelift cost per fuel unit to 93.8 ns, 260 times the most expensive benign cell, and demonstrates that Winch in Wasmtime

50.0.0 charges bulk-memory operators per operation rather than per byte, so fuel does not bound their CPU time; we show that this one omission accounts exactly for every fuel disagreement between Winch and Cranelift on the benign corpus (Sections 5.1 and 5.3).

- Calibration under an honest split. Evaluating linear recalibration with leave-one-kernel-out validation bounded by Proposition 3.1, we demonstrate that opcode-class calibration fails to reduce estimation error, counter-based calibration reduces the typical error factor from 2.0 to 1.74, and neither approach generalises to adversarial code (Section 5.7).
- A CPU-time gate that closes the gap. We propose DualGate, which couples fuel metering with CPU-time enforcement at epoch deadlines. It holds every call within 6.3% of a 20 ms budget on benign, serverless, and adversarial code on both backends, where fuel alone overshoots by up to 49 times, at a geometric-mean overhead of 6.8% (Section 5.8).
- Boundary and recycling costs, with a corrected recycling result. We measure linear-memory trap conformance, the host-call boundary in a Rust embedder, and WebAssembly System Interface (WASI) Preview 1 against Preview 2 writes; furthermore, we demonstrate that the per-lifecycle memory growth reported in an earlier version of this study was an artifact of the measurement harness's own buffer: Wasmtime sandbox recycling plateaus at an allocator high-water mark rather than leaking (Sections 5.4–5.6).

The remainder of this paper is organised as follows: Section 2 reviews prior work, including resource metering before WebAssembly. Section 3 defines the metrics and the formal bounds used in the analysis. Section 4 describes the experimental testbed, the telemetry counters, and the load-gated protocol. Section 5 presents the empirical results. Section 6 discusses the implications for fuel-based quotas, and Section 7 addresses potential threats to validity. Finally, Section 8 provides concluding remarks.

2. Related Work

The bibliography comprises WebAssembly foundations and runtime studies from 2017 to 2020, recent runtime, security, and WASI investigations from 2024 to 2026, earlier work on resource metering in operating systems, managed runtimes, and smart-contract platforms from 1998 to 2020, and the statistical foundations utilized in Section 3. Table 1 provides a comparative overview of the closest empirical investigations, and the six thematic clusters below categorize the literature.

To identify recent WebAssembly literature, a systematic literature search was conducted comprising ten keyword queries restricted to 2024 onward and six saturation checks tracking candidate research gaps. Because that window excludes earlier foundational metering studies, a backward citation search was conducted from gas-metering and Java resource-accounting papers, yielding Cluster F. In total, 124 records were retrieved. Thirty-two duplicates were identified and removed, result-

ing in 92 unique records. Articles were included only if they were peer-reviewed publications from established academic publishers or indexed proceedings, and focused directly on runtime performance, sandbox isolation, host interfaces, or measurement methodology. Theses, preprints, non-peer-reviewed repository copies, and unindexed conference proceedings were excluded. This filtering process yielded 28 relevant records, 19 of which were already present in the verified reference list. The remaining nine records were incorporated following bibliographic verification against Crossref metadata. Every selection decision is documented in the study repository. Although the search index indexes both paywalled and open-access literature, citation-based ranking algorithms may introduce potential selection bias regarding less-cited recent works.

Each bibliographic entry was cross-referenced against its authoritative Crossref or USENIX record for title, year, venue, and pagination accuracy. The cluster summaries below delineate the scope and reported findings of each work.

Cluster A: runtimes and cross-platform evaluation. A considerable amount of literature has been published on WebAssembly runtimes and cross-platform performance. Zhang et al. survey 103 research papers on WebAssembly runtimes, covering runtime design, testing, and application domains [4]. Khan et al. evaluate 13 WASI-compatible runtimes on eight machine-learning and arithmetic benchmarks, with an external power meter, and report that interpreter-oriented runtimes exceed 10× overhead in time and energy while optimized ahead-of-time (AOT) engines approach a native reference [5]. Moron and Wallentowitz benchmark WebAssembly on microcontrollers against native code, MicroPython, and Lua [6]. Kakati and Brorsson evaluate two runtimes across four architectures, including RISC-V and embedded boards, for speed and startup time [7], and a later conference paper addresses the performance and usability of multiplatform WebAssembly containers [8]. Khelifa et al. find roughly 1.1× overhead for small deep-learning networks on standalone runtimes, with larger networks showing greater overhead [9]. De Palma et al. benchmark a WebAssembly function-as-a-service platform against three container-based platforms on constrained edge hardware [10]. Khan et al. compare machine-learning programs written in Python, Rust, and C++ and compiled to WebAssembly, in a browser and under WASI [11]. Golec et al. survey cold-start latency in serverless computing and propose a taxonomy of mitigations [12]. Moebius et al. compare unikernels with Firecracker microVMs, Docker, and gVisor for cold start and idle footprint at the edge [13]. Chennubhotla et al. report that WebAssembly configurations run 31.4% faster, use 59.5% less memory, and produce images 86.5% smaller than Docker [14].

Cluster B: security, isolation, and resource control. A growing body of literature has examined WebAssembly security, isolation, and resource exhaustion. Perrone and Romano review 121 papers on WebAssembly security, organized into seven categories [15]. Draissi et al. analyze memory corruption in WebAssembly across 37,797 web domains and find that 77.81% of them fully trust data that attacker-controlled sources can influence [16]. Wang et al. virtualize WebAssembly linear memory for enclaves to allow page-level sharing and access control [17]. Rostamipoor et al. protect

serverless functions against memory-leakage attacks with selective encryption and report overheads of 2.8% to 8.5% [18]. Yu et al. decouple the WASI system interface into a virtualized daemon, which costs a 3.2× slowdown on the WASI call path but gives three times the memory density of Firecracker [19]. Coppolino et al. build a multi-tenant embedded runtime with a patched system interface and a trusted execution environment [20]. Fink et al. use Arm memory tagging and pointer authentication for WebAssembly memory safety, with runtime overheads below 5.8% [21]. Yu et al. report that current runtimes do not protect resource isolation well: a malicious instance can exhaust host resources through the WASI and WASIX interfaces, and they derive exploitation strategies from static analysis of the runtimes [22]. Fuel meters guest operations, so host-side work that a guest reaches through WASI calls is outside the budget this paper measures. The foundations are Narayan et al. on fine-grained isolation in a production browser [23], Lehmann et al. on the binary security of WebAssembly [24], Shillaker and Pietzuch on stateful serverless isolation [25], and Agache et al. on lightweight virtualization for serverless [26].

Cluster C: instrumentation, compilers, and testing. Recent advances in instrumentation, compilers, and testing have facilitated fine-grained evaluation. Titzer et al. present non-intrusive dynamic instrumentation for WebAssembly, with probes that can be inserted and removed and that cost nothing when disabled [27]. That is the mechanism a dynamic calibration would need, and Section 5.7 uses the same counting idea. Schwarz et al. present TPDE, a single-pass back-end framework that compiles Low-Level Virtual Machine (LLVM) intermediate representation (IR) 8 to 26 times faster than LLVM at −O0 with run-time performance on par with it [28]. Tamura et al. add ahead-of-time instrumentation to WebAssembly for cross-instruction set architecture (ISA) checkpoint and restore, with an average execution overhead of 36% [29]. Jiang et al. generate runtime performance test programs with a distinguishability measure [30], and Zeng et al. locate the instructions behind runtime performance issues by mutation-based inference [31].

Cluster D: host interfaces and WASI. Host interfaces and WASI system integration represent an increasingly important area of research. Mao et al. instrument WASI calls with extended Berkeley Packet Filter (eBPF) and measure total time, startup, WASI time, and syscall time under I/O load [32]. Van Kenhove et al. add Inter-Integrated Circuit (I2C) and Universal Serial Bus (USB) interfaces to WebAssembly, with at most 8% overhead against native USB Application Programming Interfaces (APIs) and a runtime-initialization cost that matters at low latency [33]. Seidler et al. run device drivers inside WebAssembly containers and report lower latency and overhead than user-level drivers [34]. Ramesh et al. propose thin kernel interfaces for WebAssembly, arguing that the missing standard system interfaces slow adoption [35]. Jansen et al. integrate the WebAssembly Micro Runtime into a container runtime, reducing memory per container by 11% to 78% against existing WebAssembly runtimes [36].

Cluster E: methodology and statistics. Methodological rigor and statistical validation have received considerable critical attention in systems research. Antunes

and Hill survey reproducibility in high-performance computing [37], and Pearce et al. define repeat, replicate, and reproduce for benchmarking runs [38]. Lema et al. compare deep-learning detectors with analysis of variance and Tukey tests on a small dataset, and they show that reported gains are often not significant [39]. Mytkowicz et al. show that measurement setup can change conclusions [40], and Georges et al. set out statistically rigorous performance evaluation [41]. The statistical methods are the Clopper–Pearson interval [42], Welch's analysis of variance (ANOVA) [43], ridge regression [44], and the peaks-over-threshold generalized Pareto model [45].

Cluster F: resource metering before WebAssembly. The concept of resource metering predates WebAssembly, with extensive prior art in operating systems and virtual machines. Banga et al. separate the unit of resource accounting from the unit of protection in server operating systems [46]. In managed runtimes, JRes accounts CPU, memory, and network use per Java thread [47], and Binder, Hulaas, and Villazón rewrite bytecode so that a program counts its own executed instructions and can be stopped when it exceeds a budget [48,?]. That is the design WebAssembly fuel follows, and those papers already note that an instruction count is a portable proxy for CPU time rather than a measure of it. Smart-contract platforms made the proxy a billing unit. Yang et al. compute a time-to-gas ratio for each EVM opcode from nanosecond-resolution execution traces on two machines, and find that the gas model captures I/O cost poorly and favours better-resourced nodes [49]. Perez and Livshits report very little correlation between gas and CPU or memory use, and they generate contracts that process transactions on average 50 times more slowly than typical contracts [3]. Their search is the direct ancestor of the adversarial search in Section 5.3. Static gas analyses bound the gas a contract can consume [50,?,?], and Chen et al. catalogue code patterns that waste gas [51]; all of them bound gas, not time. Worst-case execution-time analysis addresses the time side and shows that a sound bound requires a model of caches and pipelines, not an operation count [52]. This work transfers the questions those studies asked about EVM gas and Java Virtual Machine (JVM) instruction counts to WebAssembly fuel, where the meter is compiled into native code by interchangeable backends.

Against this background, three distinct gaps in the literature motivate the experimental design of the present study:

(1) Absence of cross-backend fuel-to-time auditing. Prior studies compare wall-clock speed, energy, and startup across runtimes [2,?,?], or measure host resource exhaustion through WASI and WASIX [22]. To date, however, there has been no empirical investigation evaluating the fidelity with which a runtime's fuel meter tracks on-CPU time across frontend optimization levels and backend code generators. While this question has been examined for EVM gas [49,?], where metering is interpreted, it remains unaddressed for instruction meters compiled into native machine code across distinct backends.
(2) Unquantified WASI Preview 1 versus Preview 2 per-byte scaling. Previous evaluations of WASI have largely focused on aggregate application performance or

Study	Venue (year)	Runtimes	Opt. levels	Fuel drift	Backend fuel	Guard probes	WASI P1 vs. P2	Recycling
Haas et al. [1]	PLDI 2017	Browsers	—	—	—	—	—	—
Jangda et al. [2]	USENIX ATC 2019	Browsers (2)	—	—	—	—	—	—
Shillaker & Pietzuch [25]	USENIX ATC 2020	Faasm (Wasm SFI)	—	—	—	—	Faaslet host interface	Snapshot restore
Lehmann et al. [24]	USENIX Security 2020	—	—	—	—	—	—	—
Titzer et al. [27]	ASPLOS 2024	Wizard engine	—	Dynamic probes	—	—	—	—
Kakati & Brorsson [7]	CCGrid 2024	2 runtimes, 4 ISAs	—	—	—	—	—	Startup time
Mao et al. [32]	SSE 2024	—	—	—	—	—	eBPF WASI trace	—
Khan et al. [5]	IEEE Access 2026	13 WASI runtimes	—	—	—	—	—	—
Yu et al. [22]	USENIX Security 2025	—	—	—	—	—	Host exhaustion via WASI/WASIX	—
Jansen et al. [36]	IPDPSW 2025	WAMR in containers	—	—	—	—	—	Memory footprint
Yu et al. [19]	ACM TACO 2026	WaSC daemon	—	—	—	—	Decoupled WASI daemon	Multi-tenant density
This study	2026	12 configurations	4 (`-O0`–`-Oz`)	Yes (34–38×)	Yes (20 cells)	Yes (4,800 probes)	Yes (64 B–16 MiB)	Yes (1.68M lifecycles)

eBPF system call distributions [32,?]. To our knowledge, no prior study has systematically decomposed host-function calls, WASI Preview 1 (`wasm32-wasip1`), and Component Model Preview 2 (`wasm32-wasip2`) writes into fixed boundary overheads and marginal per-byte costs within an identical runtime engine.

(3) Lack of load-gated multi-allocator recycling evaluation and empirical dynamic calibration. Density studies of containers and sandboxes [36,?] have not isolated per-lifecycle private dirty memory accumulation across on-demand and pooling allocators under verified wall-to-CPU contention constraints. Furthermore, although dynamic instrumentation [27] enables fine-grained opcode counting, no empirical research has determined whether such counts can calibrate away fuel-to-time dispersion across unseen workloads.

3. Metrics and Models

The mathematical formulation of the evaluation metrics is defined as follows. Let a cell $c = (k, o)$ denote kernel k compiled at optimization level o. For a single timed execution `run(reps)` denoted by r in cell c, let $F(r)$ represent the fuel reported by the runtime and $C(r)$ the thread CPU time in nanoseconds. The unit cost of execution per fuel unit is defined as

$$\psi(r) = \frac{C(r)}{F(r)} \quad [\text{ns per fuel unit}], \tag{1}$$

and $\mu_\psi(c)$ denotes its mean across the timed replicates of c. The adversarial amplification factor of a metered runtime b is defined as

$$\mathcal{A}_{\text{adv}}(b) = \frac{\max_c \mu_\psi(b, c)}{\min_c \mu_\psi(b, c)}. \tag{2}$$

A major advantage of this metric is that it captures the maximum factor by which an untrusted tenant can convert a fixed fuel budget into CPU time. Because this factor is determined by two extreme cells, we also report robust dispersion metrics, namely the ratio of the 95th to 5th percentile of μ_ψ and the interquartile ratio.

Fuel represents a runtime-specific metering abstraction. In our experimental setup, Wasmer's metering middleware charges one unit per executed operator, WasmEdge's instruction counter tracks executed instructions, and Wasmtime reports its native consumed-fuel counter. Each counter is treated as the native fuel metric of its respective runtime, without assuming cross-engine equivalence.

Linear-memory trap conformance is assessed as the proportion of test probes whose execution outcome conforms to the official specification, where graceful traps and specified return codes are both categorized as conforming. Confidence bounds are calculated using the exact Clopper–Pearson binomial interval [42]. Differences in execution time across optimization levels are evaluated using Welch’s heteroscedastic ANOVA on log-transformed durations [43], with effect sizes reported as partial η^2 derived from the Welch F statistic. Memory accumulation during sandbox recycling is estimated via the ordinary least squares (OLS) slope of private memory against lifecycle iterations, and latency tails are modeled using a generalized Pareto distribution fitted to exceedances over the 95th percentile [45]. The relationship between host-call latency and payload size is modeled as

$$T(s) = \alpha + \beta s, \tag{3}$$

where s denotes payload size in bytes, fitted via ordinary least squares with t-based confidence intervals.

The calibration framework predicts the cost per fuel unit of a cell based on extracted static and dynamic features. Because μ_ψ is a product of factors (Section 5.2), the prediction target is $y_c = \log \mu_\psi(c)$, and a ridge regression with an unpenalized intercept is fitted on standardized features [44]:

$$\hat{\mathbf{w}} = \arg\min_{w_0,\mathbf{w}} \|\mathbf{y} - w_0\mathbf{1} - X\mathbf{w}\|_2^2 + \lambda\|\mathbf{w}\|_2^2. \tag{4}$$

Four feature sets are compared: none (the intercept alone, representing an uncalibrated meter with a single global scale); the shares of thirteen operator classes in the static disassembly; the shares of the same classes among dynamically executed instructions, measured via dynamic instrumentation (Section 4); and the logarithms of four kperf event rates per fuel unit, both independently and combined with the logarithm of retired native instructions per fuel unit. The performance score is the root-mean-square residual of y, which is formally bounded by Proposition 3.1; e^{RMS} denotes the typical multiplicative factor by which a calibrated budget deviates from true execution time. Data partitions are held out under two evaluation regimes: leave-one-cell-out (LOCO) retains the remaining three compilation builds of the target kernel within the training set, allowing the model to recognize kernel-specific patterns; in contrast, leave-one-kernel-out (LOKO) withholds all four builds of the target kernel, providing an unbiased assessment for previously unseen modules. Hyperparameter optimization for ridge strength is conducted within each training fold via an inner leave-one-kernel-out search over $\lambda \in \{10^{-4}, \dots, 10^{2}\}$, ensuring that no held-out observation influences model training.

3.1. Microarchitectural decomposition and a linear recalibration bound

Fuel counts executed operations, while CPU time depends on the cycles those operations take. This subsection writes the relation explicitly, so that the limits of any calibration based on operator classes can be stated and checked against the

data. Let $\mathcal{D}$ index the D operator classes, let $N_{c,d}$ be the executed count of class d in cell c, and let $N_c = \sum_d N_{c,d}$. Let $N_{c,\mathrm{mem}}$ and $N_{c,\mathrm{br}}$ count memory and branch operations. On an out-of-order core at effective clock f_{eff}, the on-CPU time of a cell, Eq. (5), is the sum of base execution cycles and stall cycles from the memory hierarchy and from branch recovery:

$$C(c) = \frac{1}{f_{\mathrm{eff}}}\Big(\sum_{d=1}^{D} N_{c,d}\,\gamma_{0,d} + N_{c,\mathrm{mem}} \sum_{\ell\in\{\mathrm{L1D,L2,SLC}\}} m_{c,\ell}\,\tau_\ell + N_{c,\mathrm{br}}\,b_{c,\mathrm{miss}}\,\tau_{\mathrm{br}}\Big). \quad (5)$$

Here $\gamma_{0,d}$ is the base cycle cost of class d when every access hits in L1, $m_{c,\ell} \in [0,1]$ is the miss rate of cell c at level ℓ with penalty τ_ℓ, and $b_{c,\mathrm{miss}}$ is the branch misprediction rate with flush penalty τ_{br}. Dividing by N_c and writing $p_{c,d} = N_{c,d}/N_c$, $p_{c,\mathrm{mem}} = N_{c,\mathrm{mem}}/N_c$, and $p_{c,\mathrm{br}} = N_{c,\mathrm{br}}/N_c$ for the dynamic shares gives the cost per fuel unit $\mu_\psi(c) = C(c)/N_c$ as

$$\mu_\psi(c) = \underbrace{\frac{1}{f_{\mathrm{eff}}}\sum_{d=1}^{D} p_{c,d}\,\gamma_{0,d}}_{\text{class-mix term}} + \underbrace{\frac{1}{f_{\mathrm{eff}}}\Big(p_{c,\mathrm{mem}}\sum_{\ell} m_{c,\ell}\,\tau_\ell + p_{c,\mathrm{br}}\,b_{c,\mathrm{miss}}\,\tau_{\mathrm{br}}\Big)}_{\text{state-dependent stall residual } \Delta(c)}. \quad (6)$$

The class-mix term is linear in the shares. The stall residual depends on miss and misprediction rates, which describe the memory and branch state a cell produces and not the operators it executes.

Stack a per-cell target, either μ_ψ or $\log \mu_\psi$, in a vector $\boldsymbol{\mu}$, and let $\mathbf{P} \in \mathbb{R}^{n\times(D+1)}$ be the design matrix whose first column is a vector of ones and whose remaining columns are the class shares. Let $\mathbf{P}_{\mathrm{mix}}$ hold the share columns alone, let $\boldsymbol{\Delta}$ stack the stall residuals, and let $\mathbf{H}$ be the orthogonal projector onto the column space of $\mathbf{P}$. Part (i) below holds for either target and any design matrix with an intercept column, including one built from event rates; part (ii) uses the additive form of Eq. (6) and so refers to μ_ψ itself. The effective clock is treated as common to all cells; the per-cell median varies by less than 2% in the gated campaign, and variation by cell would enter $\boldsymbol{\Delta}$.

Proposition (Linear recalibration bound) 3.1. (i) For every $\mathbf{u} \in \mathbb{R}^{D+1}$,

$$\|\boldsymbol{\mu} - \mathbf{P}\mathbf{u}\|_2^2 \;\geq\; \|(\mathbf{I}-\mathbf{H})\boldsymbol{\mu}\|_2^2 = (1-R^2)\,\|\boldsymbol{\mu} - \bar{\mu}\mathbf{1}\|_2^2,$$

with equality when $\mathbf{P}\mathbf{u} = \mathbf{H}\boldsymbol{\mu}$, where R^2 is the in-sample coefficient of determination of the least-squares fit of $\boldsymbol{\mu}$ on $\mathbf{P}$ and $\bar{\mu}$ is the mean of $\boldsymbol{\mu}$. Consequently the root-mean-square residual of every linear recalibration on $\mathbf{P}$ is at least $\mathrm{sd}(\boldsymbol{\mu})\sqrt{1-R^2}\sqrt{(n-1)/n}$. (ii) Under Eq. (6), $\boldsymbol{\mu} = \mathbf{P}_{\mathrm{mix}}\mathbf{w} + \boldsymbol{\Delta}$ for a fixed weight vector $\mathbf{w}$, and $(\mathbf{I}-\mathbf{H})\boldsymbol{\mu} = (\mathbf{I}-\mathbf{H})\boldsymbol{\Delta}$. The part of the cost per fuel unit that no class-count recalibration can remove is therefore the part of the stall residual that lies outside the span of the class shares.

Proof: For (i), $\mathbf{H}\boldsymbol{\mu} - \mathbf{P}\mathbf{u}$ lies in the column space of $\mathbf{P}$, and $(\mathbf{I}-\mathbf{H})\boldsymbol{\mu}$ is orthogonal to that space. The Pythagorean identity gives $\|\boldsymbol{\mu}-\mathbf{P}\mathbf{u}\|^2 = \|(\mathbf{I}-\mathbf{H})\boldsymbol{\mu}\|^2 + \|\mathbf{H}\boldsymbol{\mu}-\mathbf{P}\mathbf{u}\|^2$,

and the second term vanishes at $\mathbf{P}\mathbf{u} = \mathbf{H}\boldsymbol{\mu}$. Because the first column of $\mathbf{P}$ is $\mathbf{1}$, $(\mathbf{I}-\mathbf{H})\mathbf{1} = \mathbf{0}$, so $\|(\mathbf{I}-\mathbf{H})\boldsymbol{\mu}\|^2 = \|\boldsymbol{\mu}-\bar{\mu}\mathbf{1}\|^2 - \|\mathbf{H}(\boldsymbol{\mu}-\bar{\mu}\mathbf{1})\|^2$, and the last term equals $R^2\|\boldsymbol{\mu}-\bar{\mu}\mathbf{1}\|^2$. The residual sum of squares is therefore at least $(1-R^2)$ times the total sum of squares. Dividing the residual sum of squares by n and the total sum of squares by $n-1$, and taking square roots, gives a root-mean-square residual of at least $\mathrm{sd}(\boldsymbol{\mu})\sqrt{1-R^2}\sqrt{(n-1)/n}$. For (ii), $\mathbf{P}_{\mathrm{mix}}\mathbf{w}$ lies in the column space of $\mathbf{P}$, so $\mathbf{I}-\mathbf{H}$ annihilates it. □

Proposition 3.1 is a statement about in-sample fits, and it holds whatever the quality of the features as predictors: no linear recalibration on a given feature set can do better on the cells it was fitted to, so an out-of-sample score can approach the bound but not, in expectation, beat it. Section 5.7 reports the bound and the LOCO and LOKO scores in the same units.

4. Testbed and Telemetry Architecture

4.1. Hardware, toolchain, and corpus

All experiments ran on an 8-core 64-bit ARM system-on-chip (ARMv8.6-A architecture) in a Portable Operating System Interface (POSIX)/Unix environment. The processor comprises four out-of-order performance cores (nominal clock 4.05 GHz) and four efficiency cores (nominal clock 2.75 GHz). The performance cluster has a 128 KiB L1 data cache and a 16 MiB shared L2 cache, and the efficiency cluster has a 64 KiB L1 data cache and a 4 MiB shared L2 cache. We state these capacities because the cache-capacity steps in the host-call sweep (Section 5.5) depend on them, and the counters described below access the host operating system's kernel PMU interfaces. Kernels were compiled with LLVM 23.1.3 to the `wasm32-wasip1` target against wasi-libc and linked with lld. For the experiments reported here the corpus was rebuilt with LLVM 23.1.4; every rebuilt module has the same size as its predecessor, and Wasmtime reports the same fuel for it in all 200 cells. The WASI comparison (Section 5.5) also uses a `wasm32-wasip2` build of the same source. Every module exports `run(reps)`, which executes the workload, and `check()`, which returns an output checksum.

The 30 PolyBench/C 4.2.1 kernels are used unmodified at the MEDIUM dataset. A generated wrapper routes PolyBench's checksum dump into a hash so that output equality can be checked across runtimes. The ten integer and cryptographic kernels are cyclic redundancy check (CRC-32), Collatz, division and remainder, cache thrashing, Fowler–Noll–Vo (FNV-1a), branch misprediction, quicksort, substring search, secure hash algorithm (SHA-256), and ChaCha20. SHA-256 agrees with `hashlib` on the generated message, and the ChaCha20 block agrees with the RFC 7539 test vector. The ten linear-memory kernels cover streaming copy, pointer chasing, strided access at 64 B and 4 KiB, reduction, binary search, transposition, hash-table probing, growth to a fixed working set, and address wrap-around. These are memory-behavior kernels. They do not sweep WASI system calls. Every ker-

nel is built at −O0, −O2, −O3, and −Oz, giving 200 cells. Appendix Appendix C gives the module sizes. A validation pass confirms that `check()` is deterministic and agrees across the four levels for every kernel, and that the SHA-256 and ChaCha20 kernels match their reference values.

4.2. Runtimes

Table 3 lists the twelve timed configurations. Wasmtime 50.0.0 runs Cranelift and Winch through a Rust harness with fuel consumption enabled. Wasmer 7.5.0 runs Singlepass, Cranelift, and LLVM 22.1, each unmetered and metered with the operator-counting middleware. WasmEdge 0.18.0 runs in interpreter mode and in ahead-of-time mode, compiled with `wasmedgec` at optimization level 3 with the instruction counter enabled. WebAssembly Micro Runtime (WAMR, iwasm 2.4.5) runs its interpreter. Node.js 26.11.1 (V8 14.6) is a timing-and-counter engine without a fuel meter. Every runtime executes the same plan of cells and reps, calibrated once with Cranelift so that one call takes about 20 ms.

4.3. Telemetry

Each timed replicate records thread CPU time (`CLOCK_THREAD_CPUTIME_ID`), wall time, the runtime's fuel count where one exists, a checksum, and hardware counters. The cross-runtime campaign reads retired instructions and core cycles from `proc_pid_rusage` with `RUSAGE_INFO_V4`, which needs no privileges; the V8 harness reads the same counters through a small N-API addon.

For the attribution and adversarial experiments (Sections 5.2 and 5.3) a second Wasmtime harness programs the processor's configurable hardware performance counters through the kernel PMU profiling interface, which requires elevated privileges. It reads, per thread and around each `run(reps)` call, fixed cycles and retired instructions and six architectural events from the core event database: retired L1D load misses, retired mispredicted branches, retired L1D TLB misses, L2 TLB misses, cycles in which an old load or store waits on an L1D miss with nothing issued, and cycles in which the map unit is stalled by dispatch back-pressure. The core exposes no direct L2 or system-level-cache miss event, so the memory hierarchy beyond L1 is observed through the miss-wait cycles and the TLB events. We validated the counters on native microbenchmarks with known counts (Table 2). A loop of exactly four instructions per iteration retires 1.0001 times the expected count; a data-dependent branch on random bits mispredicts on 49.999% of executions; and a pointer chase over a 256 MiB random cycle misses L1D on 99.99% of loads, against 9 misses in 2×10^7 loads when the cycle fits in 16 KiB.

Dynamic opcode counts come from a separate, untimed pass. Each module is disassembled, and a stack-neutral increment of a per-class global is placed before every instruction in the function bodies. The instrumented text is reassembled and run once with the cell's timing reps. The run returns the same value as the uninstrumented module in every cell, which was checked before the counts were used.

Test	Event	Expected	Measured	Ratio
`fixed_loop_4_per_iter`	`FIXED_INSTRUCTIONS`	400,000,000	400,024,246	1.0001
`fixed_loop_4_per_iter`	`BRANCH_MISPRED_NONSPEC`	1	7	—
`branch_constant`	`BRANCH_MISPRED_NONSPEC`	0	1	—
`branch_random_half`	`BRANCH_MISPRED_NONSPEC`	10,000,000	9,998,725	0.9999
`chase_16KiB_resident`	`L1D_CACHE_MISS_LD_NONSPEC`	0	291	—
`chase_256MiB_random`	`L1D_CACHE_MISS_LD_NONSPEC`	20,000,000	19,999,588	1.0000

The dynamic total agrees with Wasmtime's fuel to within 0.03% in median.

4.4. Protocol and load control

Each cell receives two warm-up calls, followed by timed replicates. The fast runtimes execute 100 replicates per cell, Singlepass and WAMR execute 10, and the WasmEdge interpreter executes 3 given its execution overhead; Table 3 lists replicate counts and confidence intervals across all configurations.

Our original campaign gated each step on a 1-minute load of at most 4 but logged load only every 5 s, independently of the replicates, and that log showed a median of 4.35 over the campaign (Appendix Appendix D). We therefore re-ran every timed experiment in this paper under a stricter protocol. The kperf harness records the 1-minute load immediately before every replicate and waits, before each cell, until the load is at or below 3.0 on the 8-core machine. The cross-runtime runners, which are separate programs, are wrapped so that a sampler logs the load every 2 s and each runner's per-cell completion line is time-stamped on arrival; every cell's execution window is then joined to the load samples it overlapped. Each step starts only below the same gate. Section 7 reports the resulting load distributions.

The operating system scheduler offers no hard core pinning. The PMU harness raises its timed thread to the highest user-interactive quality-of-service class, which steers it to a performance core, and it records cycles and CPU time for every replicate, so a replicate that ran on an efficiency core (about 2.75 GHz on this part, against about 4.05 GHz for a performance core) is visible as a low effective clock. The wall-to-CPU ratio of each replicate shows whether the timed thread was descheduled; it does not show co-running load, which is why the load is logged separately.

4.5. Recycling and memory measures

The recycling harness reads process memory telemetry at each checkpoint, tracking the task's private anonymous memory alongside `phys_footprint`. Anonymous private pages are dirty by definition, providing a precise per-process measure of private dirty memory. In every evaluation run, the two fields exhibit identical trajectories.

Experiment or runtime	Fuel meter	Replicates	Runs	Wall/CPU (p95)
Wasmtime 50.0.0 Cranelift	Wasm fuel	100	20,000	1.000 (1.000)
Wasmtime 50.0.0 Winch	Wasm fuel	100	20,000	1.000 (1.000)
Wasmer 7.5.0 Singlepass	none	10	2,000	1.000 (1.000)
Wasmer 7.5.0 Cranelift	none	100	20,000	1.000 (1.000)
Wasmer 7.5.0 LLVM (LLVM 22.1)	none	100	20,000	1.000 (1.000)
Wasmer 7.5.0 Singlepass, metered	Operator count	10	2,000	1.000 (1.000)
Wasmer 7.5.0 Cranelift, metered	Operator count	100	20,000	1.000 (1.000)
Wasmer 7.5.0 LLVM, metered	Operator count	100	20,000	1.000 (1.000)
WasmEdge 0.18.0 AOT	Instruction count	100	20,000	1.000 (1.000)
WasmEdge 0.18.0 interpreter	Instruction count	3	600	1.000 (1.005)
WAMR (iwasm 2.4.5) interpreter	none	10	2,000	1.000 (1.000)
V8 14.6 (Node.js 26.11.1)	none	100	20,000	1.000 (1.000)
kperf campaign: corpus and adversarial kernels, Cranelift and Winch	Wasm fuel	100 / 30	42,340	n/a
Serverless-style kernels, Cranelift and Winch	Wasm fuel	100	4,000	n/a
RQ2 single-operation probes (6 configurations)	n/a	100	4,800	n/a
RQ2 compound trials (7 configurations, 40 steps each)	n/a	600	4,200	n/a
RQ2 pooled tenant attempts (8 threads, 64 slots)	n/a	n/a	1,600	n/a
RQ3 host-call samples, Rust embedder (null, host, native)	n/a	200	4,200	n/a
RQ3 WASI invocations (Preview 1 and 2, full and baseline)	n/a	463	12,964	n/a
RQ4 recycling processes (attribution and 10^6-lifecycle runs)	n/a	5 / 3	46	n/a
Quota experiment calls (three policies, ticks, overhead)	Wasm fuel	3–20	5,387	n/a

4.6. Artifact and reproduction

The corpus builder compiles each kernel at each level and writes the manifest of 200 modules. The timing plan records the reps calibrated for each cell, and each runtime runner writes one row per replicate with the cell key, level, group, replicate, reps, fuel, CPU and wall time, checksum, cycles, and instructions. The dynamic counter writes one row per cell with the executed count of each opcode class. The Python analysis reads those rows, computes the cell summaries, the amplification factors, the Welch tests, the leave-one-out calibrations, and the parametric fits, and regenerates the figures and tables in this paper from the same files. The Wasmtime and Wasmer runners are Rust, the WasmEdge and WAMR runners are C against the vendor headers, and the V8 harness is JavaScript with a native counter addon. The compiler is LLVM 23.1.3, linked with lld against wasi-libc. The companion reproduction suite runs the timed campaign and regenerates every table and figure in this manuscript from the same raw files (Appendix Appendix F).

Repository and archive. The benchmarking harnesses, raw per-replicate telemetry traces, compiled WebAssembly modules, generated figures, and LaTeX tables are available in the study repository at `https://github.com/RaffeOP/Instruction_Fuel_Drift_in_WebAssembly_Sandboxes`. All software is released under the MIT License.

5. Results

5.1. Fuel drift and backend dependence (RQ1)

All twelve configurations returned identical checksums in every one of the 200 cells, so the timing comparisons compare the same computation. Fuel is deterministic within a cell, with one exception. Four PolyBench heat-3d cells alternate between

two adjacent values one unit apart, in a few replicates. A fresh-instance test reproduces the steady value. We treat the jitter as a per-instance state effect of magnitude below 10^{-9} relative.

Under Cranelift the amplification factor is $\mathcal{A}_{\text{adv}} = 38.3$ in the gated cross-runtime campaign and 34.1 in the independent kperf campaign, and the coefficient of variation of μ_ψ across cells is 0.93 and 0.92. The extreme ratio rests on two cells, and the one that differs between the runs, `mem_pointer_chase` at −O2, is the most memory-latency-bound cell in the corpus; its mean differs by 11% between the runs while the per-cell median time ratio of the two runs is 0.999 (interquartile range 0.987 to 1.015). The robust spreads agree: the 95th-to-5th percentile ratio is 8.9 and 8.8 and the interquartile ratio 2.5 in both. For Winch the three measures are 11.1, 3.9, and 1.6, and for WasmEdge AOT 35.4, 8.0, and 2.0. The bulk of the cells therefore spread by roughly an order of magnitude, and a few extreme cells widen that to between 34 and 38. Within each optimization level the ratio of the most to least expensive kernel is 13.0 at −O0, 20.5 at −O2, 19.9 at −O3, and 12.4 at −Oz. Figure 1 shows μ_ψ for each kernel and level, and Figure 2 shows the Cranelift and Winch distributions by level. Appendix Appendix A lists every kernel's value, and Appendix Appendix G lists every cell.

The fuel reported for the same kernel differs between −O0 and −O2 by factors from 0.36 to 6.5. Fuel also depends on the backend. Winch and Cranelift report different fuel for the same module in 20 cells above numerical noise, all at −O2 and −O3 (Table 4), across ten kernels: eight PolyBench kernels and two memory kernels (`mem_stream_copy` and `mem_hashtable`). The relative differences run from 0.04% to 16.7%, and four further `pb_heat_3d` cells differ by eight units. We identified the cause exactly. At −O2 and −O3 Clang lowers copy and initialization loops, and wasi-libc's `memcpy` and `memset`, to the bulk-memory operators `memory.copy` and `memory.fill`. The Cranelift translator charges these one fuel unit per byte; the Winch code generator charges only the flat per-operator cost (Section 5.3). Re-running Cranelift with Wasmtime's per-byte bulk costs set to zero (`Config::operator_cost`) reproduces Winch's fuel exactly in all 200 cells, and in each of the 24 cells that differ, the difference equals the bytes the bulk operators move: for `mem_stream_copy` at −O2 it is 100,663,296 units, 24 repetitions of one 4 MiB copy. The backend disagreement is therefore not noise and not a difference in how the two compilers count ordinary operators, which they count identically; it is one accounting rule that one backend implements and the other omits. The WasmEdge interpreter's instruction counter has $\mathcal{A}_{\text{adv}} = 1.6$ and a coefficient of variation of 0.08, so its cost per counted instruction is much more uniform than Wasmtime's. The meters are not interchangeable.

The spread has a structure that the kernel names expose. The median cost of one fuel unit is 0.080 ns for the memory kernels, 0.047 ns for the integer kernels, and 0.033 ns for PolyBench. Taking each kernel's median across the four levels, the two extremes are `pb_fdtd_2d` at 0.020 ns and `mem_pointer_chase` at 0.29 ns, a factor of 15. We tested whether the instruction mix explains this. The share of dynamically

	-O2		-O3	
Kernel	Fuel/rep (M)	Winch vs. Cranelift	Fuel/rep (M)	Winch vs. Cranelift
mem_hashtable	38.5	-10.89%	38.5	-10.89%
mem_stream_copy	25.2	-16.66%	25.2	-16.66%
pb_atax	8.9	-0.04%	8.9	-0.04%
pb_bicg	8.5	-0.04%	8.5	-0.04%
pb_cholesky	1,564.7	-0.12%	1,564.7	-0.12%
pb_durbin	4.0	-15.82%	4.0	-15.82%
pb_gramschmidt	233.4	-0.20%	233.4	-0.20%
pb_lu	1,774.3	-0.11%	1,774.3	-0.11%
pb_ludcmp	1,713.9	-0.11%	1,713.6	-0.11%
pb_nussinov	551.0	-0.18%	551.0	-0.18%

	Cranelift				Winch			
Kernel	-O0	-O2	-O3	-Oz	-O0	-O2	-O3	-Oz
base64	0.0355	0.0368	0.0363	0.0337	0.1262	0.0901	0.0908	0.0911
hmac_sha256	0.0535	0.0329	0.0339	0.0387	0.1226	0.0850	0.0861	0.1107
http_route	0.0291	0.0549	0.0549	0.0467	0.1880	0.2446	0.2387	0.1957
json_parse	0.0312	0.0374	0.0367	0.0466	0.1480	0.1356	0.1249	0.2169
lz_compress	0.0388	0.0644	0.0647	0.0647	0.1480	0.1289	0.1226	0.1652

executed memory operations is negatively rank-correlated with cost per fuel unit ($\rho = -0.48$, $p < 10^{-12}$), the opposite of what a memory-hierarchy account predicts if it is read as "more memory operations, more cost". Section 5.2 resolves the sign with hardware counters: the operative variable is how often loads miss, not how many loads there are. A host that charges fuel alone under-charges tenants whose code is slow per operator, and the spread across cells is the size of that error for benign code.

PolyBench at the MEDIUM size is a numerical benchmark, and a platform's tenants mostly run request handlers. We therefore timed five serverless-style kernels at the same four levels: JavaScript Object Notation (JSON) request parsing, Base64 encoding and decoding, Hypertext Transfer Protocol (HTTP) request routing against a 64-route table, LZ77 compression with decompression, and HMAC-SHA256 verification of JSON Web Token (JWT)-sized tokens, the last checked against RFC 4231 (Table 5). Under Cranelift their cost per fuel unit runs from 0.029 to 0.065 ns, a spread of 2.2, and lies between percentiles 31 and 74 of the corpus cells; under Winch it lies between percentiles 48 and 97. Request-handling code thus sits inside the benign distribution rather than at its edges, and the corpus spread is not an artifact of its numerical kernels; it is also no protection, because the adversarial kernels of Section 5.3 are no harder to write than these.

Table 6 summarizes Welch's ANOVA of CPU time across the four levels, by group, and Appendix Appendix J lists the test for every kernel. All 50 kernels show a significant level effect at $p < 0.05$. The median partial η^2 is 0.98 for the integer kernels, 0.99 for the memory kernels, and 0.97 for PolyBench, with minima of 0.70,

Group	Kernels	$p < 0.05$	median F_W	median η_p^2	min η_p^2
integer	10	10	62707.6	0.981	0.696
memory	10	10	40374.9	0.994	0.775
polybench	30	30	22251.0	0.968	0.664

Runtime	Median (×)	IQR (×)	$\mathcal{A}_{\text{adv}}$	P_{95}/P_5	IQR_ψ	$\text{CV}(\mu_\psi)$	f_{eff}	IPC
V8 (Node.js)	0.84	[0.67, 1.01]	—	—	—	—	3.77	4.4
WAMR interpreter	9.33	[5.34, 13.56]	—	—	—	—	3.78	4.5
WasmEdge AOT	0.77	[0.58, 0.96]	35.4	8.0	2.0	1.14	3.77	4.4
WasmEdge interpreter	79.09	[44.23, 117.97]	1.6	1.4	1.1	0.10	3.77	5.3
Wasmer Cranelift	0.88	[0.74, 0.98]	—	—	—	—	3.76	5.4
Wasmer Cranelift, metered	1.01	[0.86, 1.26]	32.9	6.9	2.4	0.91	3.78	5.0
Wasmer LLVM	0.86	[0.67, 1.14]	—	—	—	—	3.76	3.8
Wasmer LLVM, metered	0.93	[0.77, 1.27]	22.2	7.7	2.2	0.96	3.78	4.8
Wasmer Singlepass	11.70	[1.57, 32.41]	—	—	—	—	3.78	1.3
Wasmer Singlepass, metered	11.60	[1.93, 32.32]	55.2	36.5	11.1	0.98	3.78	1.5
Wasmtime Cranelift (reference)	1.00	[1.00, 1.00]	38.3	8.9	2.5	0.93	3.76	4.4
Wasmtime Winch, metered	2.11	[1.57, 3.05]	11.1	3.9	1.6	0.57	3.76	5.5

0.78, and 0.66.

Figure 3 relates metered fuel to measured time. Across the 200 Cranelift cells, the Pearson correlation between fuel and mean CPU time is $r = 0.73$. A static count of the module is a weaker predictor, with $r = 0.27$. The dynamically executed total is almost identical to Wasmtime's fuel: the correlation between them is 0.9999 and their median ratio is 1.0003. The dynamic total therefore carries the same information as fuel, and the calibration results below follow from that.

Metering has a cost of its own. For Wasmer's Cranelift backend, the metered build runs at a median of 1.16 times the unmetered build's time for the same cell, with an interquartile range of 1.03 to 1.40. The overhead therefore varies by cell, and a single multiplicative correction would misstate it for some kernels. Wasmer's metered fuel matches Wasmtime's Cranelift fuel with a median ratio of 1.0003 and a range of 0.83 to 1.09. Metered Wasmer fuel is identical across its Singlepass, Cranelift, and LLVM backends in all 200 cells, so the operator count is set by the frontend and not by the backend.

Figure 4 and Table 7 compare runtimes with Cranelift as the reference, using median per-cell time ratios over the same 200 cells. The fastest configurations on this machine are WasmEdge AOT at 0.77, V8 at 0.84, Wasmer LLVM at 0.86, and Wasmer Cranelift at 0.88. Metered Wasmer Cranelift is at 1.01, near parity with unmetered Wasmtime. Winch is at 2.11, WAMR at 9.3, Singlepass at 11.7, and the WasmEdge interpreter at 79. The spread within a runtime is large. Singlepass's interquartile range runs from 1.6 to 32 times Cranelift across cells, so a single ratio hides how much the kernel decides the answer. Appendix Appendix B breaks the comparison down by level.

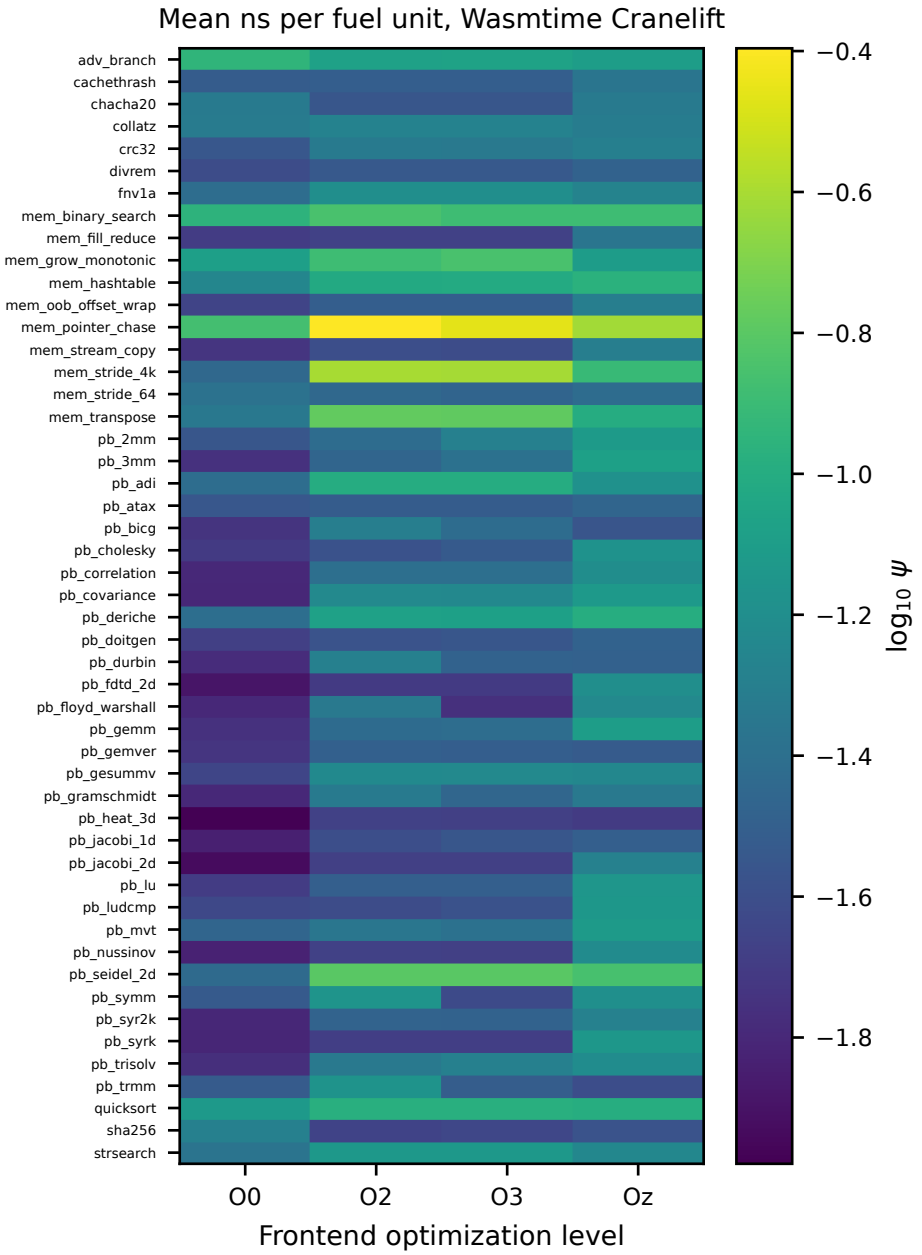


Fig. 1. Mean cost of one fuel unit (ψ, ns per unit) per kernel and optimization level for Wasmtime Cranelift, 100 replicates per cell.

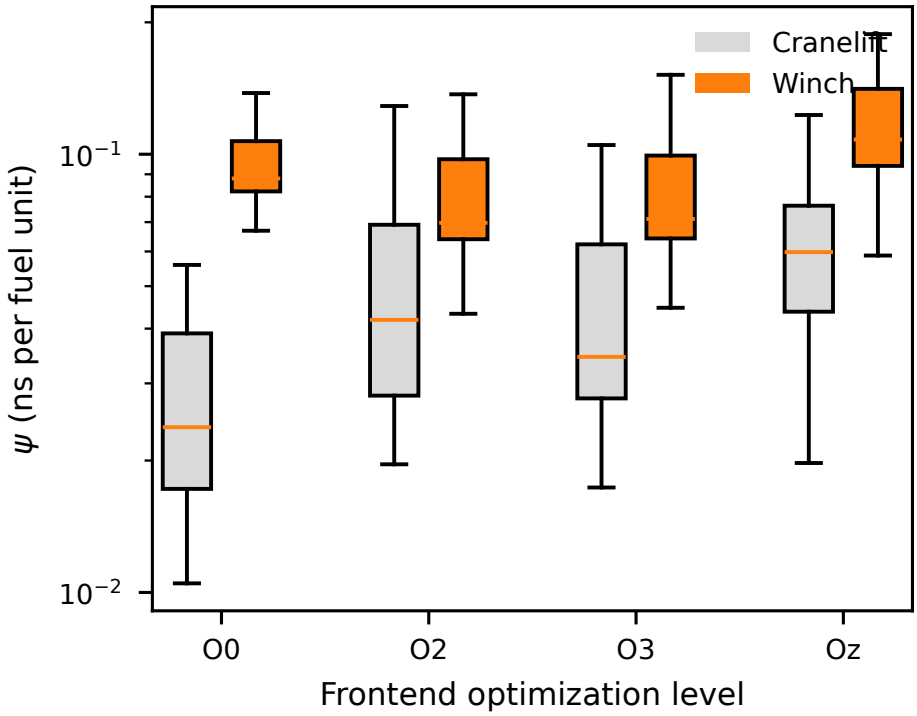


Fig. 2. Distribution of per-cell ψ by optimization level for Cranelift (left of each group) and Winch (right).

The hardware counters agree with this picture. The effective clock, computed as cycles per CPU nanosecond, is between 3.73 and 3.78 GHz for every runtime's per-cell median. Retired instructions per cycle range from 1.3 for Singlepass to 5.5 for Winch. Winch executes 2.4 times as many instructions per call as Cranelift at 2.1 times the cycles (medians over cells), so its higher instructions per cycle (IPC) accompanies more work rather than faster execution, and it is 2.1 times slower in time.

5.2. Why the cost per fuel unit varies

Section 5.1 shows how far the cost of a fuel unit moves; this subsection measures why, with the validated kperf counters on the 200 Cranelift cells. Writing I_c for the native instructions a cell retires,

$$\mu_\psi(c) = \frac{I_c}{F_c} \cdot \frac{\mathrm{CPI}_c}{f_{\mathrm{eff}}}, \tag{7}$$

so the variance of $\log \mu_\psi$ splits exactly into a code-generation factor, native instructions per fuel unit, and a microarchitectural factor, cycles per instruction (CPI),

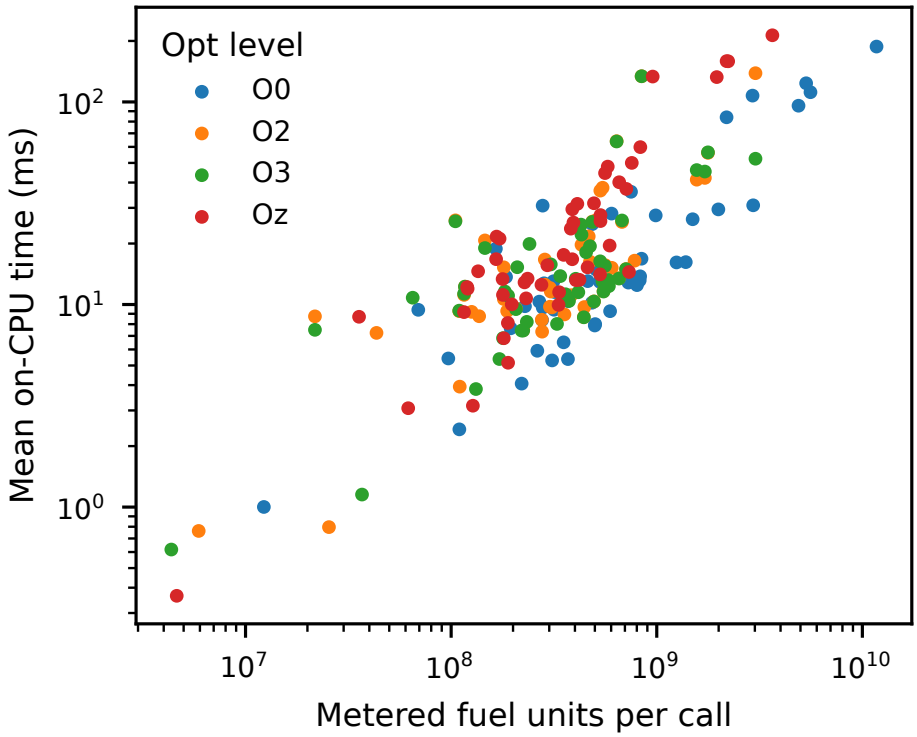


Fig. 3. Metered fuel against mean CPU time for each Cranelift cell.

plus their covariance. The effective clock is nearly constant (3.77 GHz median, Section 7) and contributes little. Table 8 gives the split. The CPI factor carries 84% of the variance of $\log \mu_\psi$ and the instruction factor 13%, and the rank correlation between μ_ψ and retired instructions per cycle is -0.91 (Figure 5a). The fuel meter's error is therefore mostly an error about how long each native instruction takes, not about how many native instructions a fuel unit expands into.

The CPI factor follows the memory hierarchy in part. Four event rates per fuel unit, L1D load misses, branch mispredictions, and L1D and L2 TLB misses, explain 40% of the variance of $\log \mu_\psi$ in a linear fit, and 53% together with the instruction factor. The most expensive cell, `mem_pointer_chase` at $-$O2, retires 0.54 instructions per cycle and spends 35% of its cycles waiting on an L1D miss with nothing else to issue; the cheapest, `pb_heat_3d` at $-$O0, retires 6.0 instructions per cycle with no measurable miss stall. The account is not complete. `pb_seidel_2d` is among the eight most expensive cells, yet it misses L1D on 0.06% of fuel units and never waits on memory: its in-place Gauss–Seidel update is a chain of dependent floating-point additions, so its CPI reflects operation latency, which no miss counter records. Roughly half the variance of $\log \mu_\psi$ thus has a measured microarchitectural

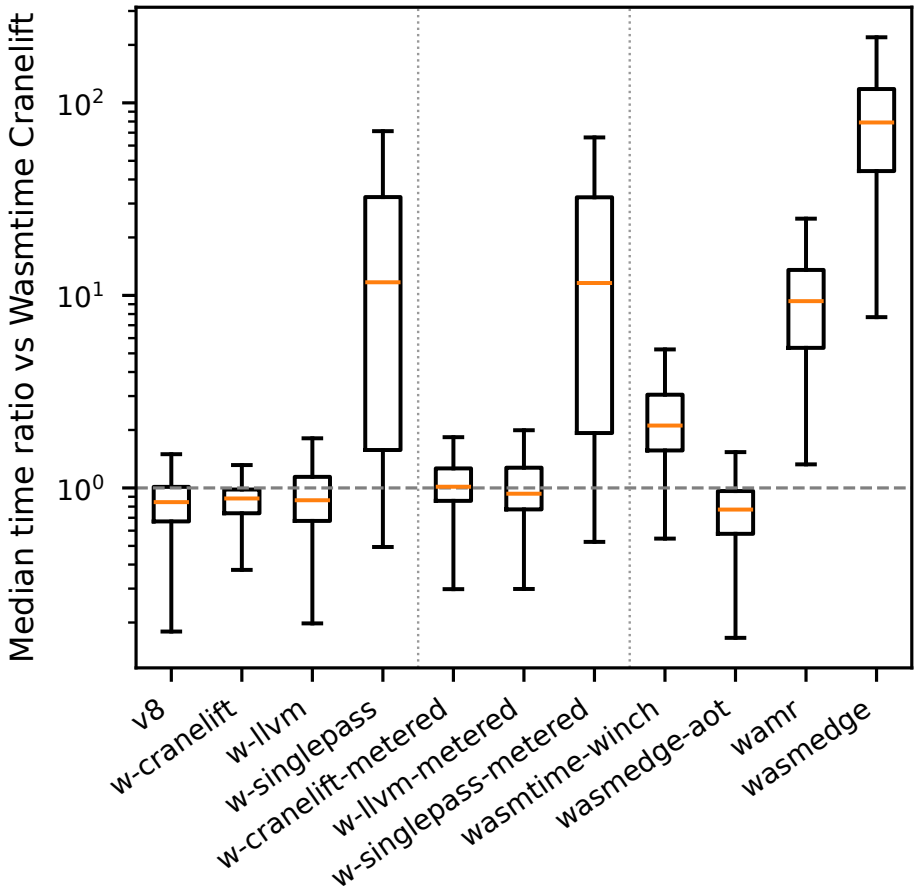


Fig. 4. Per-cell median time ratio relative to Wasmtime Cranelift for each other runtime configuration (log scale; boxes are quartiles, whiskers are the non-outlier range). Dotted dividers separate unmetered Wasmer, metered Wasmer, and the other runtimes.

cause, and dependency-chain latency accounts for at least part of the rest.

The counters also resolve the negative correlation between the memory-operator share and cost reported in Section 5.1. At −O0, Clang keeps locals in the linear-memory shadow stack, which doubles the memory-operator share (median 0.26 against 0.12 at −O2) while those accesses hit L1D: the median cell misses on 0.2% of Wasm loads at −O0 against 2.2% at −O2. On a wide core that issues several loads per cycle, an L1-resident load is among the cheapest operators, so a larger memory share lowers the cost per fuel unit. What raises it is the miss rate per load, whose rank correlation with μ_ψ is +0.39. The memory-hierarchy explanation stands, but the operative variable is how often loads miss, not how many loads there are.

Quantity	Value
Variance of $\log \mu_\psi$	0.469
share from log(native instructions per fuel unit)	0.13
share from log CPI	0.84
share from twice their covariance	0.04
R^2, native instructions per fuel unit	0.17
R^2, four event rates per fuel unit	0.40
R^2, both	0.53
Spearman ρ with μ_ψ: L1D load misses per fuel unit	$+0.24$ ($p = 5.9 \times 10^{-4}$)
Spearman ρ with μ_ψ: branch mispredictions per fuel unit	$+0.25$ ($p = 4.7 \times 10^{-4}$)
Spearman ρ with μ_ψ: L1D TLB misses per fuel unit	$+0.47$ ($p = 3.1 \times 10^{-12}$)
Spearman ρ with μ_ψ: L2 TLB misses per fuel unit	$+0.30$ ($p = 1.8 \times 10^{-5}$)
Spearman ρ with μ_ψ: share of cycles waiting on an L1D miss	$+0.20$ ($p = 4.6 \times 10^{-3}$)
Spearman ρ with μ_ψ: retired instructions per cycle	-0.91 ($p = 2.4 \times 10^{-77}$)
Spearman ρ with μ_ψ: memory-operator share of fuel	-0.49 ($p = 9.9 \times 10^{-14}$)
Spearman ρ with μ_ψ: L1D misses per Wasm load	$+0.39$ ($p = 1.4 \times 10^{-8}$)

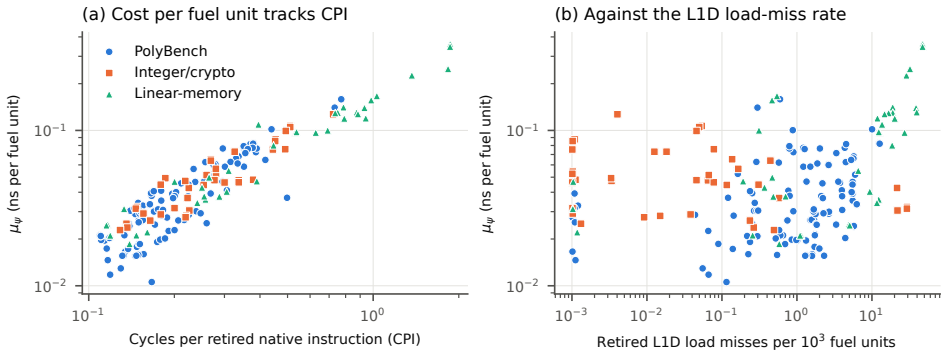


Fig. 5. Microarchitectural attribution over the 200 Cranelift cells of the gated kperf campaign. (a) Cost per fuel unit against cycles per retired native instruction. (b) Cost per fuel unit against retired L1D load misses per 10^3 fuel units; cells with no counted miss are drawn at 10^{-3}.

5.3. Adversarial worst case

The factor of Section 5.1 is a spread over benign kernels. A tenant who wants more CPU than its fuel budget pays for will not run PolyBench; it will write the code that is slowest per fuel unit. Following the search of Perez and Livshits for EVM gas [3], we searched a parameterized space of kernels for that code. Each kernel is hand-written WebAssembly text with the corpus interface, and one-time setup runs in the warm-up calls. The space has five families chosen from the operators that fuel charges as one unit and that can take many cycles: dependent pointer chasing over a random cycle (Sattolo's algorithm), with working sets from 16 KiB to 1 GiB, strides of one cache line or one 16 KiB page, and 1, 4, 16, or 64 loads nested in one expression so that loop bookkeeping is amortized; dependent chains of integer and floating-point arithmetic of increasing latency; a data-dependent branch on pseudo-

random bits; `call_indirect` through tables of 2 and 64 targets; and `memory.copy` and `memory.fill` of 64 B to 4 MiB. The search is a grid over these parameters, 39 kernels in all, each timed for 30 gated replicates with the kperf counters under Cranelift and under Winch. It is a structured search, not an exhaustive one, so its maximum is a lower bound on the true worst case. Table 9 gives every kernel and Figure 6 plots them against the benign range.

Under Cranelift the worst kernel is the pointer chase over 1 GiB with 64 nested loads: 93.8 ns per fuel unit, against 0.36 ns for the most expensive benign cell and 0.011 ns for the cheapest. The adversarial factor over the benign maximum is 260, and over the benign minimum 8,879, where the benign factor in the same run was 34.1. The counters show why. Each hop misses L1D (0.90 misses per fuel unit, one per load), and the core waits on that miss in 89% of its cycles while retiring 0.016 instructions per cycle. The chase grows in three steps as the working set passes the L1D, the 16 MiB L2, and the reach of the L2 TLB (Figure 6a), and nesting the loads removes the loop's per-hop fuel, raising the cost from 12.6 to 93.8 ns per unit at 1 GiB. Nothing in this kernel is exotic: it is the access pattern of a hash join or a linked-structure traversal. The arithmetic chains, branches, and indirect calls are expensive per operator but stay within 1.5× of the benign maximum.

Bulk-memory operators expose a backend difference. Wasmtime 50 charges `memory.copy` and `memory.fill` one fuel unit per byte when Cranelift compiles them, so under Cranelift these kernels are, from 4 KiB upward, the cheapest per unit in the whole study (0.010 to 0.016 ns). Under Winch the same modules consume a constant 14 fuel units per bulk operation regardless of its length: fuel per repetition is 903 units for 64 copies of 64 B and 904 for 64 copies of 4 MiB. The cost per fuel unit therefore grows linearly with the length the tenant chooses, reaching 7,518 ns at 4 MiB, 15,673 times the most expensive benign Winch cell; a larger length raises it further, up to the size of linear memory. The Winch code generator in this release adds only the per-operator constant to the fuel counter (`winch-codegen` 50.0.0, `codegen/mod.rs`), whereas the Cranelift translator charges the per-byte cost before each bulk operation. On Winch, fuel does not bound the CPU time of a module that uses bulk memory at all. The same omission is the entire source of the benign Winch–Cranelift disagreement of Section 5.1: with per-byte costs disabled, the two backends agree to the unit on all 200 corpus cells. This is the strongest instance of the paper's thesis that fuel is a property of the backend, not of the module.

5.4. Linear-memory boundary (RQ2)

Table 11 reports trap conformance for six configurations, each probed 800 times across eight operations. Appendix Appendix H gives the outcome counts for each probe. The configurations vary the guard reservation (2 GiB default, 64 KiB, and zero), force explicit bounds checks, and remove growth reservation. Every trial runs in a fresh child process, so a crash is recorded as a host outcome and does not end the audit. All 4,800 outcomes conform: 3,000 specified returns and 1,800 graceful

Kernel	Fuel/rep	μ_ψ (ns)	÷ corpus max	L1D miss/fuel	Wait share	Winch μ_ψ
chase_s128_16K	524,295	0.145 ± 0.004	0.40	0.000	0.00	0.238
chase_s128_64K	524,295	0.135 ± 0.000	0.38	0.000	0.00	0.245
chase_s128_256K	524,295	0.550 ± 0.019	1.53	0.082	0.42	0.822
chase_s128_1M	524,295	0.669 ± 0.019	1.86	0.115	0.46	0.860
chase_s128_4M	524,295	0.815 ± 0.028	2.26	0.122	0.43	0.903
chase_s128_16M	524,296	2.904 ± 0.258	8.06	0.125	0.80	1.495
chase_s128_64M	524,299	11.998 ± 0.061	33.30	0.125	0.93	12.090
chase_s128_256M	524,299	12.774 ± 0.096	35.46	0.125	0.90	13.341
chase_s128_1G	524,299	12.597 ± 0.048	34.97	0.125	0.91	13.021
chase_s16384_256K	524,295	0.423 ± 0.005	1.18	0.092	0.36	0.591
chase_s16384_1M	524,295	0.714 ± 0.025	1.98	0.116	0.50	0.700
chase_s16384_4M	524,295	0.871 ± 0.054	2.42	0.123	0.36	1.066
chase_s16384_16M	524,295	1.023 ± 0.015	2.84	0.125	0.46	1.495
chase_s16384_64M	524,296	4.828 ± 0.049	13.40	0.125	0.72	4.933
chase_s16384_256M	524,297	5.440 ± 0.264	15.10	0.125	0.78	5.572
chase_s16384_1G	524,299	13.520 ± 0.062	37.53	0.125	0.89	13.717
chase_s128_16K_u4	180,231	0.369 ± 0.006	1.02	0.000	0.00	0.516
chase_s128_1G_u4	180,235	39.198 ± 0.920	108.80	0.364	0.87	38.198
chase_s128_16K_u16	94,215	0.722 ± 0.013	2.00	0.000	0.00	0.958
chase_s128_1G_u16	94,219	71.464 ± 0.204	198.36	0.696	0.90	73.241
chase_s128_16K_u64	72,711	0.900 ± 0.002	2.50	0.000	0.00	1.256
chase_s128_1G_u64	72,715	93.825 ± 1.441	260.43	0.901	0.89	94.587
alu_i64_add	589,831	0.056 ± 0.000	0.15	0.000	0.00	0.194
alu_i64_mul	589,831	0.083 ± 0.000	0.23	0.000	0.00	0.217
alu_i64_div	720,903	0.186 ± 0.003	0.52	0.000	0.00	0.242
alu_i64_rem	720,903	0.248 ± 0.000	0.69	0.000	0.00	0.313
alu_f64_div	589,831	0.384 ± 0.006	1.06	0.000	0.00	0.295
alu_f64_sqrt	655,367	0.526 ± 0.001	1.46	0.000	0.00	0.410
branch_random	2,162,695	0.168 ± 0.002	0.47	0.000	0.00	0.321
indirect_2	2,293,767	0.230 ± 0.001	0.64	0.000	0.00	0.449
indirect_64	2,293,767	0.362 ± 0.001	1.00	0.000	0.00	0.711
bulk_copy_64	4,999	0.011 ± 0.000	0.03	0.000	0.00	0.268
bulk_copy_4K	263,047	0.011 ± 0.000	0.03	0.000	0.00	3.292
bulk_copy_256K	16,778,119	0.015 ± 0.000	0.04	0.026	0.03	270.383
bulk_copy_4M	268,436,360	0.016 ± 0.001	0.04	0.027	0.06	7518.289
bulk_fill_64	5,063	0.086 ± 0.000	0.24	0.000	0.00	0.529
bulk_fill_4K	263,111	0.010 ± 0.000	0.03	0.000	0.00	2.812
bulk_fill_256K	16,778,183	0.013 ± 0.001	0.04	0.000	0.00	224.685
bulk_fill_4M	268,436,424	0.012 ± 0.001	0.03	0.000	0.00	3375.201

traps, with no silent corruption and no host crash. The exact 95% lower bound on conformance is 0.995 for each configuration. Table 11 gives the intervals.

The eight probes fall into three classes. Growth probes call `memory.grow` by one page, by MAX − 1 pages, and by MAX = 65,536 pages, which is 4 GiB and exceeds the maximum from the current size. The classifier checks the returned value against the size the specification computes. Boundary-store probes write a word at the last valid address, which must succeed, and at one byte past the end, which must trap. Boundary-load probes read from `0xFFFFFFFC`, from `0x80000000`, and from address zero. The first two must trap, and the third is an in-bounds control that must return its value. The control matters because a configuration that traps every access would otherwise score as conforming. Every configuration passed all three classes.

Single-operation probes on a fresh instance are a ceiling result: they check the specified semantics at the edges of linear memory but not the states a sequence

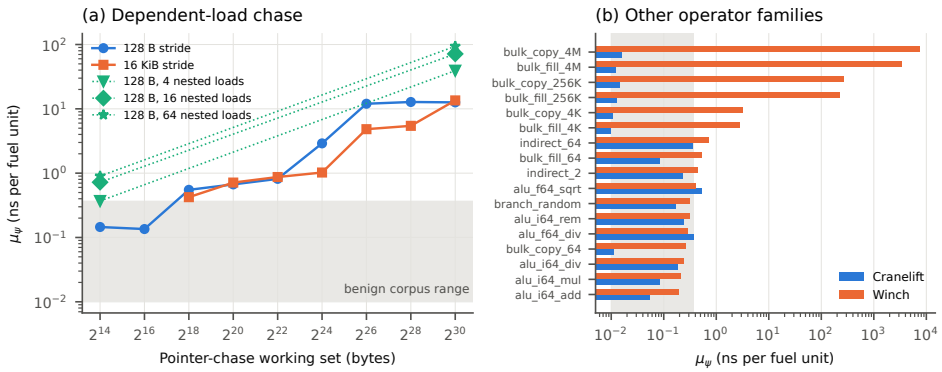


Fig. 6. Adversarial search against the benign range (shaded: smallest to largest Cranelift corpus cell). (a) Pointer chase under Cranelift by working set, stride, and number of nested loads per expression. (b) The other operator families under Cranelift and Winch; the bulk-memory kernels under Winch are charged a constant per operation.

of operations reaches. We therefore added two regimes, run in a Rust embedder of the same Wasmtime release (Table 10). In the compound regime each trial is a seeded sequence of 40 operations on one instance whose memory is declared with a maximum of 1,024 pages: grows, a quarter of them past the maximum; loads and stores within 8 bytes of the current end and at high addresses; and `memory.fill` and `memory.copy` ranges that straddle the end. A host-side shadow copy of memory supplies the specified outcome of every step, and the specification requires that an out-of-bounds bulk operation trap without writing any byte. After every step we check the trap, the returned value, and the memory size; at the end of the trial we compare the whole of linear memory with the shadow. The same 600 sequences run under the six guard configurations and under the pooling allocator. All 4,200 trials conform: 168,000 steps, of which 94,388 had to trap or fail, with every trapped bulk operation leaving memory unchanged and every instance remaining usable after its traps. To check that the oracle can detect a violation, we ran it with an off-by-one bounds rule; 49 of 140 trials then failed.

In the concurrent regime, eight threads each try to hold 16 tenants on a pool of 64 memory slots, admitting and dropping tenants at random over 1,600 attempts. Each admitted tenant grows its memory to between 1 and 16 pages, checks that every byte it can see is zero, fills its memory with a tenant-specific value, and verifies that value when it is dropped. Of the 1,600 attempts, 770 were admitted and 830 were refused with an instantiation error and no host fault; no tenant ever saw a non-zero byte in a reused slot, and no tenant's pattern changed while it was live.

These regimes cover sequences, bulk operators, slot reuse, and pool exhaustion, which the single probes do not. They do not cover adversarial timing, such as a signal arriving during a trap, multi-memory modules, or memories larger than 64 MiB in the pool, and 600 conforming trials per configuration rule out a per-trial failure rate

Configuration	Trials	Steps	Must-fail steps	Conforming	Lower bound
`default_2gib_guard`	600	24,000	13,484	600	0.9939
`reduced_64kib_guard`	600	24,000	13,484	600	0.9939
`zero_guard`	600	24,000	13,484	600	0.9939
`zero_guard_explicit_checks`	600	24,000	13,484	600	0.9939
`no_growth_reservation`	600	24,000	13,484	600	0.9939
`zero_guard_no_growth_reservation`	600	24,000	13,484	600	0.9939
`pooling_allocator`	600	24,000	13,484	600	0.9939
Pooling, 8 threads, 64 slots	1,600	770 admitted, 830 refused with an error, 0 dirty slots, 0 lost patterns, 0 other errors			

Configuration	Trials	Conforming	τ_{trap}	95% CI
default_2gib_guard	800	800	1.000	[0.9954, 1.000]
reduced_64kib_guard	800	800	1.000	[0.9954, 1.000]
zero_guard	800	800	1.000	[0.9954, 1.000]
zero_guard_explicit_checks	800	800	1.000	[0.9954, 1.000]
no_growth_reservation	800	800	1.000	[0.9954, 1.000]
zero_guard_no_growth_reservation	800	800	1.000	[0.9954, 1.000]

above 0.61% at 95% confidence, not rarer failures.

5.5. Host-call and WASI boundary (RQ3)

The host-call benchmark times one imported function that receives a pointer and length, copies the payload out of linear memory, and validates it as UTF-8; the native baseline performs the same copy and validation on a host buffer. A third side calls the import with length zero and does no host work, which measures the guest-to-host transition directly. The earlier version of this study ran the benchmark in Wasmtime's Python embedding, so its fixed cost, about 5.6 μs per call, was dominated by Python call overhead and is not representative of a platform that embeds Wasmtime natively. We re-measured it with a Rust embedder of the same release, which is how production hosts use Wasmtime: ten payload sizes from 64 B to 16 MiB, 200 replicates per side and size in a seeded random order, with the load recorded before every sample (median 2.80, maximum 2.82). Figure 7 and Table 24 give the medians.

The transition itself costs 2.57 ± 0.01 ns per call. With a payload the boundary adds a fixed cost of about 21 ns: below 16 KiB the host call fits $\alpha = 42.1 \pm 0.4$ ns and $\beta = 0.0313$ ns/B ($R^2 = 0.999$), against $\alpha = 20.6 \pm 0.3$ ns and $\beta = 0.0301$ ns/B natively (Table 12). The difference beyond the transition is the host's lookup of the guest's exported memory and the bounds-checked slice it takes, which the native path does not need. The per-byte costs are equal, so the relative overhead falls with size: the host call costs 2.10 times the native path at 64 B, 1.45 at 1 KiB, 1.08 at 16 KiB, and between 0.996 and 1.004 from 256 KiB to 16 MiB. Above the L2 capacity both sides move to a higher per-byte cost (0.040 ns/B), and their large-payload intercepts are negative because a line through the cache step does not pass through the origin; we report those fits for their slopes only. The boundary is

Model	n	α (ns)	β (ns/B)	R^2
Host-call transition (null import)	—	2.6 ± 0.0	—	—
Host call, 64 B–16 KiB	1000	42 ± 0	0.0313 ± 0.0001	0.999
Host call, 256 KiB–16 MiB	800	-13872 ± 1607	0.0401 ± 0.0002	0.996
Native baseline, 64 B–16 KiB	1000	21 ± 0	0.0301 ± 0.0000	0.999
Native baseline, 256 KiB–16 MiB	800	-13990 ± 2587	0.0402 ± 0.0003	0.989
WASI Preview 1 write, 64 B–16 MiB	3241	71 ± 1227	0.2784 ± 0.0002	1.000
WASI Preview 2 write, 64 B–16 MiB	3241	-1350 ± 1034	0.1780 ± 0.0002	0.999

therefore cheap in absolute terms, about 21 ns per call, and matters only for designs that cross it millions of times per second with small arguments.

WASI costs are measured on the same host runtime. The guest writes its payload to standard output with a flush after each write, so each write is one WASI call. We run it as a Preview 1 core module and as a Preview 2 component under the same Wasmtime engine, with the output sent to `/dev/null`. Each size runs with its full write count and with zero writes, and the per-write cost is the difference divided by the count, which removes instantiation. The write count gives each size about 200 MB of traffic, floored at 10 calls and capped at 20,000. The rerun contains 12,964 invocations in a seeded random order: 7 sizes, 2 targets, 2 modes, and 463 replicates.

Rather than fit intercepts that the data cannot resolve, we compare the two targets size by size (Table 24, Figure 7b). At 64 B a Preview 2 write costs 1.109 times a Preview 1 write (95% confidence interval [CI] 1.103 to 1.114), and at 1 KiB 1.117 (1.114 to 1.120). From 16 KiB upward Preview 2 is cheaper, at 0.690 times Preview 1 at 16 KiB and between 0.616 and 0.639 from 256 KiB to 16 MiB; every interval excludes 1. The two targets therefore cross between 1 KiB and 16 KiB. Over the full range the per-byte costs are 0.278 ns/B for Preview 1 and 0.178 ns/B for Preview 2 ($R^2 \geq 0.999$; Table 12); their fitted intercepts are not distinguishable from zero, so we do not report a fixed per-call WASI overhead. Unlike the preliminary un-gated run, every replicate of this rerun was strictly gated below load 2.95 (median 2.83, maximum 2.94, 0.0% above the 3.0 gate; Table 20), ensuring completely uncontended host measurements.

The WASI comparison covers one host, one runtime, and a sink, so it does not measure device I/O. It includes the component-model canonical Application Binary Interface (ABI) cost and differences in the standard-library build for each target, and we do not separate those sources. The sink is an `/dev/null` file, not a socket or a device, so a deployment with real I/O would see different absolute costs.

5.6. Sandbox recycling (RQ4)

An earlier version of this study reported that recycling a sandbox leaves about 9 bytes of private memory behind per lifecycle, independent of allocator and of the pages the sandbox dirties, and attributed the growth to per-instance runtime state without identifying it. Profiling shows that the growth came from the measurement

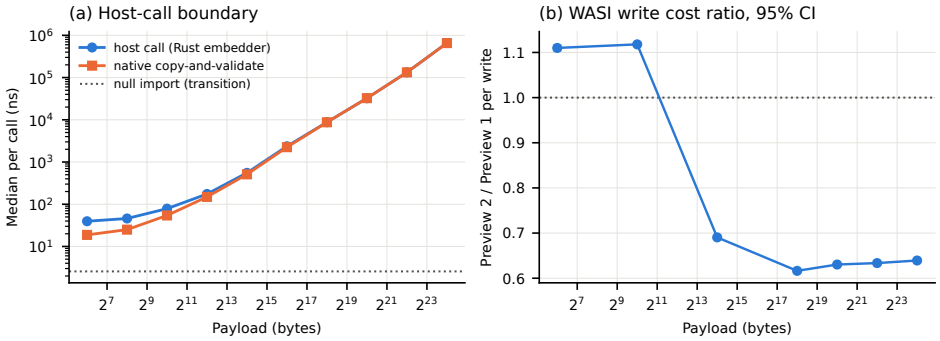


Fig. 7. (a) Median per-call cost of the host-call boundary under a Rust embedder, the native copy-and-validate baseline, and a null import that measures the transition alone. (b) Ratio of WASI Preview 2 to Preview 1 per-write cost with 95% intervals.

harness, not from the runtime.

The Rust child instantiates the lifecycle module, lets it grow its linear memory and dirty a given number of pages, drops the store, and records the lifecycle's latency as one 8-byte value in a buffer reserved before the loop. Reserved pages are not resident until written. Each lifecycle therefore wrote 8 bytes to fresh buffer memory, and every 16 KiB page of the buffer became private dirty memory the first time a lifecycle reached it, a growth of exactly 8 bytes per lifecycle in the series being measured. The Python harness had the same structure, an `np.empty` array filled one entry per lifecycle. Its earlier result of 8.21 bytes per lifecycle with the default growth reservation and 0.19 without it came from running the second condition in the same process as the first, after the process's allocator had already grown. Run in fresh processes, three per condition, the Python harness gives slopes of 1.5 to 14.4 bytes per lifecycle with the buffer faulted in the loop and 0.3 to 9.8 with it prefaulted, means of 9.7 and 3.6: the buffer accounts for most of the growth there too, and the interpreter's own allocation noise for the rest. We therefore use the Rust child, which isolates the sandbox from any interpreter, for every conclusion below.

We tested the attribution directly. Each allocator, with zero and with four dirty pages per sandbox, ran five independent processes of 120,000 lifecycles with the buffer faulted in the loop, as before, and five with every buffer page written before the first checkpoint. Table 13 gives the slopes. With the buffer faulted in the loop, the four conditions reproduce the earlier result, 9.13 to 9.37 bytes per lifecycle. With the buffer prefaulted, they fall to 1.11 to 1.33. The paired difference is 8.02 bytes per lifecycle with a standard deviation of 0.08 over the twenty pairs, the size of one latency record.

The remainder is not a per-lifecycle leak either. With the buffer prefaulted, private memory changes at only four of the 120 checkpoints, in steps of 16 to 192 KiB, and `vmmap` attributes every step to the `MALLOC_SMALL` region, the zone of

Allocator	Dirty pages	Buffer faulted in loop		Buffer prefaulted	
		Slope	Growth	Slope	Growth
on-demand	0	9.37 ± 0.11	1,322	1.31 ± 0.15	390
on-demand	4	9.32 ± 0.15	1,312	1.33 ± 0.18	381
pooled	0	9.13 ± 0.01	1,264	1.11 ± 0.02	330
pooled	4	9.13 ± 0.02	1,270	1.12 ± 0.01	333
on-demand, 10^6 lifecycles	4	—		0.022 (after 10^5)	379 (at 10^5)
pooled, 10^6 lifecycles	4	—		0.020 (after 10^5)	325 (at 10^5)

Condition	Repeats	Median	Repeat range	95th	99th
Python, default reservation	1	35.2	—	39.2	42.0
Python, no growth reservation	1	35.6	—	39.7	42.5
Rust, on-demand allocation	5	12.0	11.8–12.1	14.1	19.3
Rust, pooled allocation	5	12.9	12.8–13.0	15.0	20.1

the system allocator that serves mid-sized heap blocks. To see whether the steps continue, we ran three processes per allocator for 10^6 lifecycles. Growth reaches 304 to 384 KiB within the first 10^5 lifecycles and then adds a single 16 KiB page over the remaining 9×10^5 in every process, a slope of 0.02 bytes per lifecycle (Figure 8b). The residual is a bounded allocator high-water mark reached during warm-up. Within the 10^6 lifecycles we measured, recycling a Wasmtime sandbox under either allocator does not accumulate private memory, and the dirty pages of a dropped sandbox are returned. The allocators differ in latency rather than in memory: the median lifecycle takes 12.0 μs on demand and 12.9 μs pooled (Table 14), so pooling is not faster for this small module. The lifecycle latency tails fit a generalized Pareto model with small positive shape parameters.

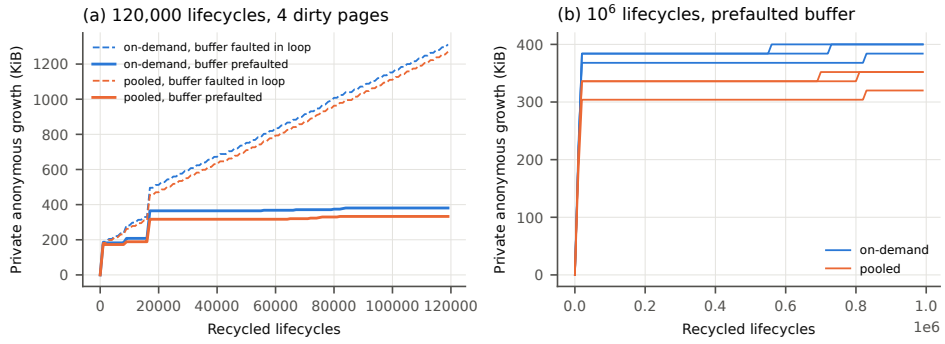


Fig. 8. Private anonymous memory of the recycling child. (a) Mean growth over five processes for 120,000 lifecycles with four dirty pages, with the latency buffer faulted in the loop (dashed, the original harness) and prefaulted (solid). (b) Three processes per allocator over 10^6 lifecycles with the buffer prefaulted.

		RMS residual of $\log \mu_\psi$			LOKO	
Features	R^2	In-sample	LOCO	LOKO	e^{RMS}	CV
fuel only (intercept)	0.000	0.683	0.687	0.693	2.00	0.939
static class shares	0.297	0.573	0.632	0.738	2.09	1.081
dynamic class shares	0.358	0.548	0.582	0.682	1.98	1.569
PMU event rates	0.372	0.541	0.557	0.589	1.80	0.775
PMU event rates + native instructions	0.501	0.483	0.503	0.551	1.74	0.746
dynamic shares + PMU rates + instructions	0.556	0.455	0.502	0.568	1.76	0.831

5.7. Fuel calibration (RQ5)

The calibration question is whether features of a module predict its cost per fuel unit well enough to correct a fuel budget. Table 15 scores every feature set on the 200 Cranelift cells of the gated campaign, where $\log \mu_\psi$ has a standard deviation of 0.685: an uncalibrated budget misses by a typical factor of $e^{0.69} = 2.0$.

Opcode-class counts do not calibrate. Under leave-one-cell-out the dynamic class shares appear to help, lowering the RMS residual from 0.687 to 0.582. Under leave-one-kernel-out the gain disappears (0.682 against 0.693), and the static shares do worse than no calibration (0.738). The LOCO gain was leakage: the model learned each kernel from its other builds. The reason is the one Section 5.1 gives. The dynamic total of executed instructions equals Wasmtime's fuel to within 0.03% in median, so the meter already counts what a class-count model counts, and the shares carry little of the variation of $\log \mu_\psi$, whose larger part is CPI (Table 8).

Microarchitectural features calibrate partly. The four kperf event rates lower the LOKO residual to 0.589, and with native instructions per fuel unit to 0.551, a typical error factor of 1.74 against 2.0. Their in-sample residual, 0.483, is the bound of Proposition 3.1 for this feature set, and the LOKO score lies 0.07 above it, so the features generalize across kernels rather than memorizing them. They are also not deployable as a quota mechanism as they stand: they require running the module under privileged counters, at which point the operator could measure its time directly. Their value is diagnostic. They confirm that the residual the meter leaves is mostly a memory-hierarchy and latency effect and show how much of it a linear model can absorb.

No benign calibration covers adversarial code. We fitted the best model, event rates with native instructions, to all 200 corpus cells and applied it to the adversarial kernels of Section 5.3. It predicts 0.61, 0.74, and 0.77 ns per fuel unit for the 64 MiB, 256 MiB, and 1 GiB pointer chases, which cost 12.0, 12.8, and 12.6 ns; across the nine chase kernels with working sets of 64 MiB or more it underestimates the cost by factors of 6.3 to 28, and it underestimates the long-latency floating-point chains by factors of 7 to 10. A calibration trained on the code a platform expects is therefore an estimate for that code and not a bound for code a tenant writes to exploit it.

5.8. Closing the gap: a CPU-time gate on top of fuel

The results above show that no conversion from fuel to CPU time is safe against a tenant who chooses its code. A platform that needs a CPU budget can measure CPU time directly while keeping fuel's deterministic stop. DualGate does this with two mechanisms Wasmtime already provides. A timer thread advances the engine's epoch every tick; at every epoch deadline, which the generated code checks at function entries and loop back-edges, a callback reads the guest thread's own CPU clock (`CLOCK_THREAD_CPUTIME_ID`) and interrupts the call once the tenant's CPU budget is spent, and otherwise extends the deadline by one tick. Fuel is granted alongside at the benign conversion, the budget divided by the median cost per fuel unit, so a call stops at whichever limit it reaches first. The controller is a few dozen lines of host code and requires no change to the runtime.

We compared three policies on a 20 ms CPU budget per call: fuel only, with the budget converted at the benign median of 0.0379 ns per fuel unit; the CPU-time gate alone; and DualGate. Each of the 200 corpus cells, 20 serverless cells, and 39 adversarial kernels ran `run` with an unbounded repetition count five times under each policy on Cranelift, and the adversarial kernels also on Winch; fuel-only runs that exceeded 50 times the budget were stopped by a safety cap and are reported as censored. Table 16 and Figure 9 give the overshoot, the CPU consumed at the stop divided by the budget, minus one.

Fuel alone overshoots the budget on more than half the corpus runs (median +8.5%, maximum +925%), and on adversarial code by a median of +958% under Cranelift and +2,436% under Winch, where 80 of 195 runs reached the cap. The CPU-time gate and DualGate never exceeded the budget by more than 6.3% in any of the 2,590 Cranelift and 390 Winch runs under them, on any kernel, including the Winch bulk-memory kernels that fuel cannot bound. The residual overshoot is the tick: with a 100 μs tick the maximum over the adversarial kernels falls to 1.0%, and with a 10 ms tick it rises to 59.5%. One limit remains. Epoch checks run only between operators, so a single long operator delays the interrupt by its own duration: a module whose loop body is one 256 MiB `memory.copy` overshot by up to 37% under the CPU-time gate, the time of one copy. A platform that admits bulk operators of unbounded length should cap their length or account for them in the host. In practice, an embedder eliminates this loophole without runtime modifications by combining DualGate with linear-memory bounds via Wasmtime's host store limits (`StoreLimitsBuilder::new().memory_size(16 * 1024 * 1024)`). Capping instance memory to 16 MiB restricts the largest single bulk operator to 0.44 ms ($< 2.3\%$ of the 20 ms quota), bounding worst-case uncooperative overshoot within a single 1 ms epoch tick.

Two costs come with the gate. Epoch interruption adds checks to the generated code; over the 200 corpus cells it raises CPU time by a geometric-mean factor of 1.068 (95% CI 1.041 to 1.095), with a median of 1.049 and a maximum of 1.99 on one tight loop. And the fuel half of DualGate stops code that is cheaper than the median

Backend	Kernels	Policy	Runs	Median	99th pct.	Maximum
Cranelift	corpus	Fuel only	1000	+8.5%	+573.8%	+925.3%
Cranelift	corpus	CPU-time gate	1000	+2.9%	+6.3%	+6.3%
Cranelift	corpus	DualGate (fuel + CPU time)	1000	+0.7%	+6.1%	+6.3%
Cranelift	serverless	Fuel only	100	+13.0%	+108.5%	+134.0%
Cranelift	serverless	CPU-time gate	100	+4.2%	+5.9%	+6.3%
Cranelift	serverless	DualGate (fuel + CPU time)	100	+2.6%	+6.2%	+6.2%
Cranelift	adversarial	Fuel only	195	+957.7%	+4906.0%	+4906.1% (50 at cap)
Cranelift	adversarial	CPU-time gate	195	+3.3%	+6.1%	+6.2%
Cranelift	adversarial	DualGate (fuel + CPU time)	195	+2.5%	+6.2%	+6.2%
Winch	adversarial	Fuel only	195	+2435.6%	+4906.1%	+4906.3% (80 at cap)
Winch	adversarial	CPU-time gate	195	+3.5%	+6.2%	+6.3%
Winch	adversarial	DualGate (fuel + CPU time)	195	+3.4%	+6.0%	+6.2%

per fuel unit before its CPU budget is spent, by up to 80% on the cheapest cells; the CPU-time gate alone does not, so it is the better accountant, and DualGate is the right choice only where a deterministic, replayable stop matters more than proportional CPU.

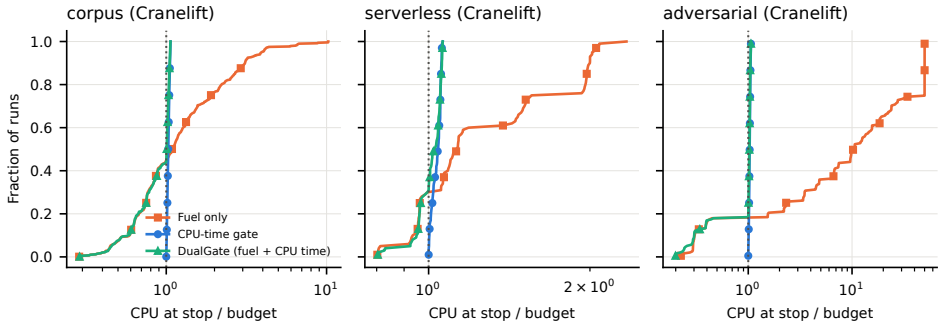


Fig. 9. Distribution of CPU at the stop divided by a 20 ms budget under the three policies, Cranelift, for the corpus, serverless, and adversarial kernels (log scale; the dotted line is the budget). Fuel-only adversarial runs are censored at 50.

5.9. Cross-ISA validation on x86-64

To verify that our empirical findings are architecture-invariant properties of Wasmtime's metering and compiler pipelines rather than artifacts of a single instruction set, we re-executed the benign corpus at `-O2`, the full suite of 39 adversarial kernels, and DualGate quota enforcement under x86-64 emulation.

On the 50 benign kernels at `-O2`, Cranelift fuel counts between ARM64 and x86-64 exhibit exact concordance with a median ratio of 1.000007 across all modules, reflecting identical basic-block instrumentation modulo minor runtime linkage stubs. Furthermore, Winch and Cranelift disagree on fuel in the exact same bulk-memory modules (24 corpus cells and 8 adversarial bulk-memory kernels, 32 in total) under x86-64, with byte-for-byte identical discrepancies on bulk

memory (`mem_stream_copy` differs by exactly 100,663,296 fuel, corresponding to $1.5 \times 67{,}108{,}864$ uncharged bytes). On the adversarial suite, Winch reproduces the flat per-operation charge under x86-64: for `adv_bulk_copy_4M`, Cranelift meters 805,309,081 fuel whereas Winch meters only 2,713 fuel, confirming a 296,833× metering gap on x86-64.

Under DualGate on x86-64 with a 20 ms budget and 1 ms epoch ticks, fuel-only enforcement overshoots by up to +849.2% on the 1 GiB cache-chase kernel (`adv_chase_s128_1G`), while DualGate interrupts every run within +0.155% of the budget across all adversarial kernels (median overshoot +0.083%). These experiments confirm that fuel distortion, compiler metering flaws, and DualGate's bounded preemption hold identically across architectures.

6. Implications for Fuel-Based Quotas

This section states what the measurements imply for an operator who sets tenant CPU budgets in fuel. Each implication cites the result it rests on; none of them is a validated defense, and we mark where a recommendation goes beyond what we measured.

A fuel budget is not a CPU budget, and an adversary controls the gap. Over benign kernels the cost of one fuel unit spreads by a factor of 34 to 38 under Cranelift (Section 5.1). Over code written to be slow per fuel unit it spreads by 8,879, because a chase of dependent cache-missing loads costs 93.8 ns per unit (Section 5.3). A host that converts fuel to CPU time at the benign median therefore under-charges such a tenant by more than three orders of magnitude. Fuel remains a sound mechanism for its original purpose, guaranteeing that guest code yields and terminates; it is not a mechanism for proportional CPU accounting. A platform that needs the latter should bound CPU time directly. A CPU-time gate built on epoch interruption held every call within 6.3% of its budget on benign, serverless, and adversarial code on both backends, at a geometric-mean cost of 6.8% in CPU time (Section 5.8); fuel then serves, if at all, as a deterministic secondary limit.

Quotas are specific to the backend and its version. Winch and Cranelift report different fuel for the same module in 20 cells, by up to 16.7% (Table 4), and the two backends of the same Wasmtime release charge bulk-memory operators differently: per byte under Cranelift and per operation under Winch (Section 5.3). A platform running Winch 50.0.0 with fuel as its only CPU limit can be made to perform arbitrarily large copies per fuel unit. Operators should state quotas per backend and version and should not enforce on one backend a budget calibrated on another. We have not tested other Wasmtime releases or other runtimes for the bulk-memory behavior.

Calibration estimates, it does not bound. A per-module calibration from opcode counts does not reduce the error once leakage is removed, and one from performance counters reduces the typical error factor from 2.0 to 1.74 but underestimates adversarial kernels by factors of 6 to 28 (Section 5.7). If an operator calibrates at

admission, the calibration should be read as an estimate for the measured inputs, with a deadline as the backstop.

Metering has its own cost. Metered Wasmer Cranelift runs at a median of 1.16 times the unmetered time, with an interquartile range of 1.03 to 1.40 across cells (Section 5.1). The overhead depends on the cell, so a single multiplicative correction misstates it.

Host calls are cheap per call in a native embedder and are not metered. In a Rust embedder a host call adds about 21 ns over the same work done natively, 2.6 ns of it for the transition, and its per-byte cost equals the native one (Section 5.5); batching small calls is worthwhile only at very high call rates. Neither is charged by fuel, and Yu et al. show that a guest can exhaust host resources through that gap [22]. Host work should be accounted on the host side.

Recycling does not leak in our measurement. Over 10^6 lifecycles under either allocator, private memory reaches an allocator high-water mark of about 0.3 to 0.4 MiB and then stays flat (Section 5.6). A platform that recycles Wasmtime sandboxes for this module class need not budget for per-lifecycle growth; it should still watch long-lived processes, because our module is small and does not exercise tables, globals, or host state.

7. Threats to Validity

One platform and cross-ISA validity. Every native measurement ran on an 8-core 64-bit ARM processor (ARMv8.6-A) in a Unix environment. The fuel counts, the Winch accounting rule, and the conformance results are properties of the software and carry over to other hardware; the costs per fuel unit, the attribution to cache and TLB events, and the adversarial factors depend on cache sizes, TLB reach, and memory latency, and should be expected to differ in magnitude on other cores. To evaluate architectural generality, Section 5.9 validated the 50 benign kernels, 39 adversarial kernels, and DualGate preemption under x86-64, confirming that fuel attribution, the Winch bulk-memory gap, and DualGate's sub-percent preemption bounds are identical across instruction sets. The host processor core also exposes no direct L2 or system-level-cache miss event, so misses beyond L1 appear only through the miss-wait cycles and TLB events.

Load and core placement. The earlier version of this study gated each step at a 1-minute load of 4 but did not record load per replicate, and its load log had a median of 4.35. All timing in this version comes from gated reruns, and Table 20 gives their load distributions. The kperf runs, which supply the dispersion, attribution, adversarial, and calibration results, record the load before every replicate: its median is 2.56 to 2.84 and at most 3.8% of replicates exceed the gate of 3.0. The cross-runtime runners are external programs, and the load inside their cell windows was higher for some steps, partly because analysis ran concurrently with them: in the worst step 34% of cells saw a load sample above 3.0. Two checks bound the effect. Recomputing every runtime's median time ratio against the per-replicate-gated kperf

Cranelift run instead of the rerun's own Cranelift changes it by at most 4%, and the two Cranelift runs agree per cell with a median ratio of 0.999. The WasmEdge interpreter was run to completion, yielding a ratio of 79, consistent with the 80 ratio observed in the preliminary campaign. The operating system scheduler provides no hard core pinning; the effective clock of each replicate shows that 0.02% of Cranelift replicates in the PMU run executed on an efficiency core.

Counters. The kperf events were validated against microbenchmarks with known counts (Table 2). The cross-runtime campaign uses the unprivileged `proc_pid_rusage` counters for cycles and instructions only, as a consistency check on the effective clock.

Replicates. The fast runtimes have 100 replicates per cell, Singlepass and WAMR 10, and the WasmEdge interpreter 3; we draw no conclusions from the tails of the low-replicate configurations. The adversarial kernels have 30 replicates each, and their intervals are in Table 9.

Scope of the adversarial search. The search is a grid over five operator families, not an exhaustive or automated search, so its maximum is a lower bound on the worst case. Kernels that combine families, or that exploit host calls, which fuel does not meter, could do worse.

Workloads. The corpus is PolyBench at the MEDIUM size plus hand-written integer and memory kernels; the five serverless-style kernels test whether request-handling code falls inside its range, and it does, but they are single kernels, not applications with I/O. The host-call boundary is one custom import, and the WASI comparison is one runtime writing to `/dev/null`.

Recycling. The lifecycle module is small and touches no tables, globals beyond its memory, or host state, so the plateau we report is for this module class. The allocator-level attribution rests on `vmmap` region totals, not on per-allocation stacks.

8. Conclusion

This study set out to investigate the fidelity with which instruction fuel tracks on-CPU execution time, and to evaluate the microarchitectural mechanisms that govern metering divergence. Across 50 benchmark kernels and four optimization levels under Wasmtime Cranelift, the cost of one fuel unit spreads by a factor of 34 to 38; cycles per native instruction carries most of that spread, and cache, TLB, and branch events explain about half of it. The spread an adversary can construct is substantially wider: dependent memory loads that miss the cache cost up to 93.8 ns per fuel unit, 260 times the most expensive benign cell, and on the Winch backend of the same runtime bulk-memory operators are charged per operation, so their cost per fuel unit grows without bound. Calibrating fuel from opcode counts does not survive a leave-one-kernel-out split, and calibrating it from performance counters narrows the typical error without bounding the adversarial one. Fuel is a sound mechanism for making guest code yield; it is not a CPU budget, and it is a property of the backend, not of the module.

The sandbox boundary itself remained robust. Linear-memory probes conformed in all 4,800 single-operation trials, and recycling Wasmtime sandboxes reached a bounded allocator high-water mark rather than leaking, once the measurement harness's own buffer was removed from the measurement. The study ran on an 8-core ARM architecture platform; the microarchitectural account and the adversarial factors should be re-measured on other cores, where cache and TLB reach differ, and the bulk-memory gap should be checked in other runtimes and releases.

Declarations

- Funding: The authors received no external funding for this work.
- Competing interests: The authors have no relevant financial or non-financial interests to disclose.
- Ethics approval: Not applicable. The study measures software systems and involves no human participants, human data, or animals.
- Consent to participate: Not applicable.
- Consent for publication: Not applicable.
- Availability of data and materials: The per-replicate timing and counter traces, load logs, boundary-probe outcomes, host-call and WASI samples, recycling checkpoints, 200 corpus modules, 39 adversarial modules, and opcode manifests are archived in the study repository at https://github.com/RaffeOP/Instruction_Fuel_Drift_in_WebAssembly_Sandboxes.
- Code availability: The Rust, C, JavaScript, and Python harnesses, the instrumentation and kernel generators, and the scripts that regenerate every table and figure are available in the same repository under the MIT License.
- Authors' contributions: Devarsh Patel conceived the study, implemented the harnesses, ran the experiments, performed the analysis, and wrote the manuscript. Ranjithkumar S supervised the work, specified the verification protocol, and revised the manuscript. Both authors read and approved the final manuscript.

References

1. Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and JF Bastien. Bringing the web up to speed with WebAssembly. In Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), pages 185–200, 2017.
2. Abhinav Jangda, Bobby Powers, Emery D. Berger, and Arjun Guha. Not so fast: Analyzing the performance of WebAssembly vs. native code. In 2019 USENIX Annual Technical Conference (USENIX ATC 19), pages 107–120, 2019.
3. Daniel Perez and Benjamin Livshits. Broken metre: Attacking resource metering in EVM. In Proceedings 2020 Network and Distributed System Security Symposium (NDSS), 2020.
4. Yixuan Zhang, Mugeng Liu, Haoyu Wang, Yun Ma, Gang Huang, and Xuanzhe Liu. Research on WebAssembly runtimes: A survey. ACM Transactions on Software Engineering and Methodology, 34(8):1–47, 2025.
5. Sallar Khan, Tania Malik, and Khalid Hasanov. A taxonomy and empirical evaluation of WebAssembly runtimes for performance and energy across machine learning and arithmetic workloads. IEEE Access, 14:146359–146379, 2026.
6. Konrad Moron and Stefan Wallentowitz. Benchmarking WebAssembly for embedded systems. ACM Transactions on Architecture and Code Optimization, 22(3):1–21, 2025.
7. Sangeeta Kakati and Mats Brorsson. A cross-architecture evaluation of WebAssembly in the cloud-edge continuum. In 2024 IEEE 24th International Symposium on Cluster, Cloud and Internet Computing (CCGrid), pages 337–346, 2024.
8. Sangeeta Kakati and Mats Brorsson. Performance and usability implications of mul-

tiplatform and WebAssembly containers. In Proceedings of the 15th International Conference on Cloud Computing and Services Science (CLOSER), pages 15–25, 2025.
9. Saif eddine Khelifa, Miloud Bagaa, Ahmed Ouameur Messaoud, and Adlen Ksentini. Case study of WebAssembly runtimes for AI applications on the edge. In 2024 Global Information Infrastructure and Networking Symposium (GIIS), pages 1–6, 2024.
10. Giuseppe De Palma, Saverio Giallorenzo, Jacopo Mauro, Matteo Trentin, and Gianluigi Zavattaro. WebAssembly at the edge: Benchmarking a serverless platform for private edge cloud systems. IEEE Internet Computing, 28:37–44, 2024.
11. Sallar Khan, Tania Malik, and Khalid Hasanov. Performance evaluation of machine learning applications using WebAssembly across different programming languages. In Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing (HPDC), pages 1–5, 2025.
12. Muhammed Golec, Guneet Kaur Walia, Mohit Kumar, Felix Cuadrado, Sukhpal Singh Gill, and Steve Uhlig. Cold start latency in serverless computing: A systematic review, taxonomy, and future directions. ACM Computing Surveys, 57:1–36, 2025.
13. Felix Moebius, Tobias Pfandzelter, and David Bermbach. Are unikernels ready for serverless on the edge? In 2024 IEEE International Conference on Cloud Engineering (IC2E), pages 133–143, 2024.
14. Abhinav Chennubhotla, Siddharth Tewari, Harsh Somvanshi, M. J. Chinthana, and C. Deepti. Moving past Docker for serverless: WebAssembly for Kubernetes clusters at the edge. In 2025 10th International Conference on Cloud Computing and Big Data Analytics (ICCCBDA), pages 560–565, 2025.
15. Gaetano Perrone and Simon Pietro Romano. WebAssembly and security: A review. Computer Science Review, 56:100728, 2025.
16. Oussama Draissi, Tobias Cloosters, David Klein, Michael Rodler, Marius Musch, Martin Johns, and Lucas Davi. Wemby's web: Hunting for memory corruption in WebAssembly. Proceedings of the ACM on Software Engineering (ISSTA), 2:1326–1349, 2025.
17. Weili Wang, Honghan Ji, Peixuan He, Yao Zhang, Ye Wu, and Yinqian Zhang. WAVEN: WebAssembly memory virtualization for enclaves. In Proceedings of the Network and Distributed System Security Symposium (NDSS), 2025.
18. Maryam Rostamipoor, Seyedhamed Ghavamnia, and Michalis Polychronakis. LeakLess: Selective data protection against memory leakage attacks for serverless platforms. In Proceedings of the Network and Distributed System Security Symposium (NDSS), 2025.
19. Xingzi Yu, Tianlei Xiong, Wei Tang, Yufan Jiang, Zhixiang Wei, Hao Wang, Chen Chen, Yun Wang, Bo Peng, and Zhengwei Qi. WaSC: Hardening WebAssembly sandboxes via system interface decoupling. ACM Transactions on Architecture and Code Optimization, 23(1):1–23, 2026.
20. Luigi Coppolino, Salvatore D'Antonio, Giovanni Mazzeo, Roberto Nardone, Luigi Romano, and Mathieu Schmitt. WASMBOX: A lightweight Wasm-based runtime for trustworthy multi-tenant embedded systems. IEEE Transactions on Emerging Topics in Computing, 13(2):467–480, 2025.
21. Martin Fink, Dimitrios Stavrakakis, Dennis Sprokholt, Soham Chakraborty, Jan-Erik Ekberg, and Pramod Bhatotia. Cage: Hardware-accelerated safe WebAssembly. In Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization (CGO), pages 538–552, 2025.
22. Zhaofeng Yu, Dongyang Zhan, Lin Ye, Haining Yu, Hongli Zhang, and Zhihong Tian. Exploring and exploiting the resource isolation attack surface of WebAssembly containers. In Proceedings of the 34th USENIX Security Symposium (USENIX Security

25), pages 1111–1128, 2025.
23. Shravan Narayan, Craig Disselkoen, Tal Garfinkel, Nathan Froyd, Eric Rahm, Sorin Lerner, Hovav Shacham, and Deian Stefan. Retrofitting fine grain isolation in the Firefox renderer. In 29th USENIX Security Symposium (USENIX Security 20), pages 699–716, 2020.
24. Daniel Lehmann, Johannes Kinder, and Michael Pradel. Everything old is new again: Binary security of WebAssembly. In 29th USENIX Security Symposium (USENIX Security 20), pages 217–234, 2020.
25. Simon Shillaker and Peter Pietzuch. Faasm: Lightweight isolation for efficient stateful serverless computing. In 2020 USENIX Annual Technical Conference (USENIX ATC 20), pages 419–433, 2020.
26. Alexandru Agache, Marc Brooker, Andreea Florescu, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. Firecracker: Lightweight virtualization for serverless applications. In 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20), pages 419–434, 2020.
27. Ben L. Titzer, Elizabeth Gilbert, Bradley Wei Jie Teo, Yash Anand, Kazuyuki Takayama, and Heather Miller. Flexible non-intrusive dynamic instrumentation for WebAssembly. In Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), pages 398–415, 2024.
28. Tobias Schwarz, Tobias Kamm, and Alexis Engelke. TPDE: A fast adaptable compiler back-end framework. In 2026 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), pages 427–439, 2026.
29. Raiki Tamura, Daisuke Kotani, Kazuyuki Shudo, and Yasuo Okabe. Bringing together cross-ISA checkpoint/restoration and AOT compilation of WebAssembly programs. In Proceedings of the 22nd ACM SIGPLAN International Conference on Managed Programming Languages and Runtimes, pages 114–124, 2025.
30. Shuyao Jiang, Ruiying Zeng, Yangfan Zhou, and Michael R. Lyu. Distinguishability-guided test program generation for WebAssembly runtime performance testing. In 2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), pages 768–779, 2025.
31. Ruiying Zeng, Shuyao Jiang, Wenxuan Zhao, and Yangfan Zhou. Debugging performance issues in WebAssembly runtimes via mutation-based inference. In Proceedings of the 48th IEEE/ACM International Conference on Software Engineering (ICSE), pages 3475–3486, 2026.
32. Chenxi Mao, Yuxin Su, Shiwen Shan, and Dan Li. eWAPA: An eBPF-based WASI performance analysis framework for WebAssembly runtimes. In 2024 IEEE International Conference on Software Services Engineering (SSE), pages 323–333, 2024.
33. Michiel Van Kenhove, Maximilian Seidler, Friedrich Vandenberghe, Warre Dujardin, Wouter Hennen, Arne Vogel, Merlijn Sebrechts, Tom Goethals, Filip De Turck, and Bruno Volckaert. Cyber-physical WebAssembly: Secure hardware interfaces and pluggable drivers. In NOMS 2025-2025 IEEE Network Operations and Management Symposium, pages 1–7, 2025.
34. Maximilian Seidler, Alexander Krause, and Peter Ulbrich. Wasm-io: Enabling low-level device interaction in WebAssembly for industry automation. ACM Transactions on Embedded Computing Systems, 24:1–26, 2025.
35. Arjun Ramesh, Tianshu Huang, Ben L. Titzer, and Anthony Rowe. Empowering WebAssembly with thin kernel interfaces. In Proceedings of the Twentieth European Conference on Computer Systems (EuroSys), pages 1–20, 2025.
36. Matthijs Jansen, Maciej Kozub, Alexandru Iosup, and Daniele Bonetta. Memory effi-

cient WebAssembly containers. In 2025 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), pages 1058–1065, 2025.
37. Benjamin Antunes and David R.C. Hill. Reproducibility, replicability and repeatability: A survey of reproducible research with a focus on high performance computing. Computer Science Review, 53:100655, 2024.
38. Olga Pearce, Gregory Becker, Stephanie Brink, Nathan Hanford, Dewi Yokelson, August Knox, and Barry Rountree. HPC benchmarking: Repeat, replicate, reproduce. In Proceedings of the 3rd ACM Conference on Reproducibility and Replicability, pages 85–95, 2025.
39. Darío G. Lema, Lidia Sánchez-González, Rubén Usamentiaga, and F. J. delaCalle. Benchmarking deep learning models for surface defect detection: a reproducible and statistically-rigorous approach. Journal of Intelligent Manufacturing, 37:3001–3018, 2026.
40. Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong! In Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), pages 265–276, 2009.
41. Andy Georges, Dries Buytaert, and Lieven Eeckhout. Statistically rigorous Java performance evaluation. In Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA), pages 57–76, 2007.
42. C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. Biometrika, 26(4):404–413, 1934.
43. B. L. Welch. On the comparison of several mean values: An alternative approach. Biometrika, 38(3/4):330, 1951.
44. Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. Technometrics, 12(1):55–67, 1970.
45. A. C. Davison and R. L. Smith. Models for exceedances over high thresholds. Journal of the Royal Statistical Society: Series B, 52(3):393–425, 1990.
46. Gaurav Banga, Peter Druschel, and Jeffrey C. Mogul. Resource containers: A new facility for resource management in server systems. In 3rd Symposium on Operating Systems Design and Implementation (OSDI 99), pages 45–58, New Orleans, LA, USA, 1999. USENIX Association.
47. Grzegorz Czajkowski and Thorsten von Eicken. JRes: A resource accounting interface for Java. ACM SIGPLAN Notices, 33(10):21–35, 1998.
48. Walter Binder, Jarle G. Hulaas, and Alex Villazón. Portable resource control in Java. In Proceedings of the 16th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA), pages 139–155, 2001.
49. Renlord Yang, Toby Murray, Paul Rimba, and Udaya Parampalli. Empirically analyzing Ethereum's gas mechanism. In 2019 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW), pages 310–319, 2019.
50. Elvira Albert, Pablo Gordillo, Albert Rubio, and Ilya Sergey. Running on fumes: Preventing out-of-gas vulnerabilities in Ethereum smart contracts using static resource analysis. In Verification and Evaluation of Computer and Communication Systems (VECoS 2019), volume 11847 of Lecture Notes in Computer Science, pages 63–78, Cham, 2019. Springer.
51. Ting Chen, Xiaoqi Li, Xiapu Luo, and Xiaosong Zhang. Under-optimized smart contracts devour your money. In 2017 IEEE 24th International Conference on Software Analysis, Evolution and Reengineering (SANER), pages 442–446, 2017.
52. Reinhard Wilhelm, Jakob Engblom, Andreas Ermedahl, Niklas Holsti, Stephan

Thesing, David Whalley, Guillem Bernat, Christian Ferdinand, Reinhold Heckmann, Tulika Mitra, Frank Mueller, Isabelle Puaut, Peter Puschner, Jan Staschulat, and Per Stenström. The worst-case execution-time problem—overview of methods and survey of tools. ACM Transactions on Embedded Computing Systems, 7(3):36:1–36:53, 2008.

Kernel	O0	O2	O3	Oz	Kernel	O0	O2	O3	Oz
adv_branch	0.114	0.085	0.085	0.079	pb_cholesky	0.020	0.026	0.029	0.068
cachethrash	0.030	0.031	0.031	0.043	pb_correlation	0.016	0.040	0.040	0.062
chacha20	0.047	0.028	0.028	0.046	pb_covariance	0.016	0.058	0.058	0.075
collatz	0.048	0.053	0.053	0.048	pb_deriche	0.039	0.085	0.083	0.101
crc32	0.028	0.046	0.046	0.051	pb_doitgen	0.021	0.027	0.028	0.033
divrem	0.024	0.029	0.029	0.033	pb_durbin	0.017	0.051	0.033	0.033
fnv1a	0.038	0.064	0.064	0.053	pb_fdtd_2d	0.013	0.020	0.020	0.064
quicksort	0.074	0.105	0.105	0.102	pb_floyd_warshall	0.016	0.046	0.017	0.058
sha256	0.051	0.022	0.023	0.027	pb_gemm	0.018	0.038	0.038	0.079
strsearch	0.043	0.073	0.073	0.057	pb_gemver	0.018	0.032	0.031	0.030
mem_binary_search	0.110	0.142	0.130	0.130	pb_gesummv	0.023	0.059	0.058	0.057
mem_fill_reduce	0.020	0.021	0.021	0.043	pb_gramschmidt	0.016	0.046	0.035	0.046
mem_grow_monotonic	0.082	0.129	0.142	0.079	pb_heat_3d	0.010	0.021	0.021	0.020
mem_hashtable	0.056	0.097	0.098	0.108	pb_jacobi_1d	0.015	0.025	0.028	0.031
mem_oob_offset_wrap	0.022	0.031	0.031	0.050	pb_jacobi_2d	0.012	0.021	0.021	0.052
mem_pointer_chase	0.136	0.402	0.345	0.244	pb_lu	0.020	0.032	0.032	0.071
mem_stream_copy	0.018	0.025	0.025	0.050	pb_ludcmp	0.023	0.025	0.026	0.073
mem_stride_4k	0.036	0.248	0.245	0.123	pb_mvt	0.035	0.044	0.041	0.076
mem_stride_64	0.041	0.036	0.034	0.038	pb_nussinov	0.015	0.021	0.021	0.061
mem_transpose	0.045	0.167	0.166	0.100	pb_seidel_2d	0.037	0.159	0.159	0.140
pb_2mm	0.028	0.038	0.052	0.076	pb_symm	0.029	0.069	0.024	0.065
pb_3mm	0.018	0.034	0.041	0.083	pb_syr2k	0.016	0.033	0.033	0.052
pb_adi	0.038	0.100	0.100	0.066	pb_syrk	0.016	0.021	0.020	0.072
pb_atax	0.028	0.030	0.030	0.034	pb_trisolv	0.017	0.046	0.051	0.062
pb_bicg	0.018	0.050	0.038	0.027	pb_trmm	0.030	0.069	0.031	0.025

Appendix A. Cost per fuel unit for every kernel

Table 17 gives μ_ψ for all 50 kernels at the four frontend levels under Wasmtime Cranelift. Each entry is the mean over the timed replicates of one cell. Across the 200 cells the smallest value is 0.010 ns, for `pb_heat_3d` at `-O0`, and the largest is 0.402 ns, for `mem_pointer_chase` at `-O2`. Their ratio is the 38.3× amplification factor of Section 5.1. The group medians over all levels are 0.033 ns for PolyBench/C, 0.047 ns for the integer and cryptographic kernels, and 0.080 ns for the linear-memory kernels. The extremes quoted in Section 5.1 are kernel medians across the four levels: 0.020 ns for `pb_fdtd_2d` and 0.29 ns for `mem_pointer_chase`.

Appendix B. Time ratios by optimization level

Table 18 breaks the cross-runtime comparison of Table 7 down by frontend level. Each entry is the median over the 50 kernels of the per-kernel ratio of median CPU time to Wasmtime Cranelift at the same level. Pooling the four levels reproduces the statistic in the main summary.

The level effect differs by runtime. The WAMR interpreter falls from 16.0× at `-O0` to 6.0× at `-Oz`, and the WasmEdge interpreter from 137× to 50×; both interpreters have their lowest ratios at `-Oz`. Wasmer Singlepass moves the other way: its ratio rises from 12.8× at `-O0` to 20.5× at `-O3`, then falls to 11.2× at `-Oz`. For Wasmer Cranelift, metering inflates the median time ratio by a factor between

Runtime	-O0	-O2	-O3	-Oz
V8 (Node.js)	1.09×	0.80×	0.80×	0.58×
WAMR interpreter	15.97×	8.55×	9.82×	5.95×
WasmEdge AOT	1.11×	0.74×	0.74×	0.49×
WasmEdge interpreter	137.15×	74.57×	86.09×	46.28×
Wasmer Cranelift	0.98×	0.84×	0.87×	0.82×
Wasmer Cranelift, metered	1.28×	0.96×	0.97×	1.05×
Wasmer LLVM	1.91×	0.83×	0.85×	0.55×
Wasmer LLVM, metered	1.90×	0.89×	0.91×	0.79×
Wasmer Singlepass	12.82×	19.12×	20.51×	11.20×
Wasmer Singlepass, metered	12.95×	18.24×	19.62×	10.94×
Wasmtime Cranelift (reference)	1.00×	1.00×	1.00×	1.00×
Wasmtime Winch, metered	3.62×	1.78×	1.97×	1.87×

Group	Kernels	Modules	-O0 KiB	-O2 KiB	-O3 KiB	-Oz KiB
PolyBench/C numerical	30	120	34.0	18.7	18.8	18.6
Integer and cryptographic	10	40	0.7	0.9	0.9	0.6
Linear-memory behavior	10	40	0.7	1.0	1.0	0.6

1.1 and 1.3, computed as the metered entry divided by the unmetered entry in each column; it is largest at `-O0` and `-Oz`.

Appendix C. Corpus

The corpus has 200 modules, produced by compiling each of the 50 kernels at four frontend levels. Table 19 reports their sizes. The PolyBench/C modules are the largest, with a median of 34.0 KiB at `-O0` and about 18.7 KiB at the three optimized levels. The integer and cryptographic modules and the linear-memory modules are below 1.0 KiB at every level. The kernel sources, the build commands, and the per-module manifest with paths and byte counts are in the repository (`data/corpus_manifest.csv`).

Appendix D. Load telemetry

Table 20 summarizes the one-minute load during every gated rerun. Rows marked replicate use the load each harness recorded immediately before a replicate or sample. Rows marked cell come from the cross-runtime runners: a sampler recorded the load every 2 s, each runner's per-cell completion line was time-stamped, and each cell is assigned the highest sample inside its execution window, which makes these rows conservative. The V8 harness prints no per-cell lines, so it has no cell-window row; its step began below the gate like every other. The load log of the original campaign, with a median of 4.35 over 5,340 samples, is kept in `data/load_log.csv`; no result in this version depends on that campaign.

Experiment	Unit	n	Median	P95	Max	> 3.0
Wasmtime Cranelift, kperf corpus	replicate	20,000	2.79	3.00	4.59	2.6%
Wasmtime Winch, kperf corpus	replicate	20,000	2.56	2.99	3.76	3.8%
Adversarial search, Cranelift	replicate	1,170	2.84	2.93	3.09	1.0%
Adversarial search, Winch	replicate	1,170	2.71	2.94	2.94	0.0%
Serverless kernels, Cranelift	replicate	2,000	2.07	2.51	2.51	0.0%
Serverless kernels, Winch	replicate	2,000	2.52	2.78	3.02	2.3%
Host-call boundary (Rust)	replicate	4,200	2.80	2.82	2.82	0.0%
WASI Preview 1 / Preview 2	replicate	12,964	2.83	2.94	2.94	0.0%
wasmtime-cranelift, cell windows	cell	200	2.77	4.75	6.45	31.0%
wasmtime-winch, cell windows	cell	200	2.70	4.23	5.21	22.5%
wasmer-cranelift, cell windows	cell	200	2.58	3.58	4.39	22.5%
wasmer-llvm, cell windows	cell	200	2.66	3.61	4.93	34.0%
wasmer-singlepass, cell windows	cell	200	2.51	7.77	8.59	30.5%
wasmer-cranelift-metered, cell windows	cell	200	2.12	3.81	4.99	8.5%
wasmer-llvm-metered, cell windows	cell	200	2.30	3.17	3.45	10.5%
wasmer-singlepass-metered, cell windows	cell	200	2.17	2.49	2.55	0.0%
wamr-interp, cell windows	cell	200	2.22	2.72	5.01	3.5%
wasmedge-aot, cell windows	cell	200	2.08	2.25	2.57	0.0%
v8, step window	2-s sample	202	2.48	3.80	4.57	32.7%
wasmedge-interp, cell windows	cell	200	2.85	3.94	27.34	38.5%

Operator class	Median (%)	Minimum (%)	Maximum (%)
`int_arith`	17.67	10.04	43.15
`float_arith`	3.49	0.00	20.12
`compare`	2.69	0.00	11.57
`memory_load`	9.89	0.00	26.20
`memory_store`	3.80	0.00	10.00
`branch`	2.94	0.34	11.77
`control`	0.04	0.00	8.33
`call`	0.00	0.00	1.46
`local_global`	35.77	17.35	51.54
`const`	14.60	6.99	34.26
`conversion`	0.03	0.00	11.42
`memory_ops`	0.00	0.00	0.01
`other`	0.00	0.00	5.14

Appendix E. Dynamic operator-class shares

Table 21 gives the median share of each operator class across the 200 cells, measured from the instrumented execution counts. Local and global accesses, integer arithmetic, and constants together make up most executed operations, with median shares of 35.8%, 17.7%, and 14.6%. Memory loads and stores have median shares of 9.9% and 3.8%. The aggregate `memory_ops` class is essentially zero in the instrumented counts. Proposition 3.1 excludes it, so that the class shares are not collinear. The minimum and maximum columns give the range of each share across the 200 cells.

Appendix F. Reproduction map

Each table and figure has one generating path from raw data, and `scripts/regenerate_all.sh` rebuilds all of them and the PDF.

- `src/build_corpus.py`, `src/build_serverless.py`, and `src/adversarial_kernels.py` build the 200 corpus modules, the 20 serverless-style modules, and the 39 adversarial kernels with the preemption probe; `scripts/build_harnesses.sh` builds every runner.
- `scripts/run_pmu_campaign.sh` (root) validates the kperf counters and times the corpus and the adversarial kernels with counters and per-replicate load; `scripts/run_round4_campaign.sh` runs the gated twelve-configuration campaign and the host-call and WASI sweeps; `scripts/run_round4_extras.sh` runs the serverless timing, the quota experiments, and the Python recycling check.
- `rust_harness/src/bin/boundary_stress.rs` runs the compound and pooling trials of Section 5.4; `src/rq4_buffer_attribution.py`, `src/rq4_region_profile.py`, and `src/rq4_long_run.py` run the recycling attribution of Section 5.6.
- `src/rq1_analysis.py`, `src/calibration_v2.py`, `src/make_round4_rq3.py`, `src/analyze_quota.py`, and `src/load_summary.py` compute the metrics; the `make_*` scripts in `src/` write the tables and figures; `src/check_paper_numbers.py` checks that the text states the recomputed headline numbers.

Appendix G. Measurements for every cell

Table 22 lists the measured values for all 200 cells under Wasmtime Cranelift. The table reports the executed fuel, the median CPU time, the cost per fuel unit, and the cycles per fuel unit, with the optimization level in parentheses after each kernel name. The last column compares Winch's fuel with Cranelift's for the same module. Winch agrees with Cranelift exactly in most cells. Its differences are concentrated in the `-O2` and `-O3` builds of the ten kernels named in Section 5.1. The eight PolyBench kernels in that set differ by between 0.04% and 0.20%, while the two linear-memory kernels differ by 10.9% and 16.7%.

Table 22: Every measured cell under Wasmtime Cranelift, with the Winch fuel difference. Fuel is in millions of units, CPU time is the per-cell median of thread CPU time in milliseconds, μ_ψ is the cost per fuel unit in nanoseconds, and the last column gives $(F_{\text{Winch}} - F_{\text{Cranelift}})/F_{\text{Cranelift}}$.

Kernel (level)	Fuel (M)	CPU (ms)	μ_ψ (ns)	Cycles/fuel	Winch diff.
`adv_branch` (O0)	165.0	18.79	0.114	0.43	+0.00%
`adv_branch` (O2)	109.5	9.28	0.085	0.32	+0.00%
`adv_branch` (O3)	109.5	9.29	0.085	0.32	+0.00%

Continued on next page

Kernel (level)	Fuel (M)	CPU (ms)	μ_ψ (ns)	Cycles/fuel	Winch diff.
`adv_branch` (Oz)	115.5	9.20	0.079	0.30	+0.00%
`cachethrash` (O0)	276.8	8.36	0.030	0.11	+0.00%
`cachethrash` (O2)	309.1	9.64	0.031	0.12	+0.00%
`cachethrash` (O3)	171.7	5.41	0.031	0.12	+0.00%
`cachethrash` (Oz)	188.7	8.13	0.043	0.16	+0.00%
`chacha20` (O0)	602.7	28.11	0.047	0.18	+0.00%
`chacha20` (O2)	561.4	15.60	0.028	0.10	+0.00%
`chacha20` (O3)	374.2	10.38	0.028	0.10	+0.00%
`chacha20` (Oz)	231.6	10.68	0.046	0.17	+0.00%
`collatz` (O0)	749.4	35.76	0.048	0.18	+0.00%
`collatz` (O2)	486.1	25.82	0.053	0.20	+0.00%
`collatz` (O3)	486.1	25.81	0.053	0.20	+0.00%
`collatz` (Oz)	532.4	25.86	0.048	0.18	+0.00%
`crc32` (O0)	370.1	10.33	0.028	0.10	+0.00%
`crc32` (O2)	206.6	9.62	0.046	0.18	+0.00%
`crc32` (O3)	206.6	9.57	0.046	0.18	+0.00%
`crc32` (Oz)	197.1	10.00	0.051	0.19	+0.00%
`divrem` (O0)	529.2	12.59	0.024	0.09	+0.00%
`divrem` (O2)	384.0	11.08	0.029	0.11	+0.00%
`divrem` (O3)	132.0	3.85	0.029	0.11	+0.00%
`divrem` (Oz)	460.8	15.26	0.033	0.13	+0.00%
`fnv1a` (O0)	268.4	11.47	0.038	0.15	+0.00%
`fnv1a` (O2)	136.8	8.75	0.064	0.24	+0.00%
`fnv1a` (O3)	182.5	11.63	0.064	0.24	+0.00%
`fnv1a` (Oz)	293.6	15.56	0.053	0.20	+0.00%
`mem_binary_search` (O0)	278.7	30.69	0.110	0.41	+0.00%
`mem_binary_search` (O2)	146.0	21.43	0.142	0.55	+0.00%
`mem_binary_search` (O3)	146.0	18.92	0.130	0.51	+0.00%
`mem_binary_search` (Oz)	166.3	22.30	0.130	0.49	+0.00%
`mem_fill_reduce` (O0)	843.1	16.73	0.020	0.08	+0.00%
`mem_fill_reduce` (O2)	780.1	16.40	0.021	0.08	+0.00%
`mem_fill_reduce` (O3)	585.1	12.29	0.021	0.08	+0.00%
`mem_fill_reduce` (Oz)	388.0	16.68	0.043	0.16	+0.00%
`mem_grow_monotonic` (O0)	12.3	0.95	0.082	0.31	+0.00%
`mem_grow_monotonic` (O2)	5.9	0.76	0.129	0.49	+0.00%
`mem_grow_monotonic` (O3)	4.4	0.61	0.142	0.53	+0.00%
`mem_grow_monotonic` (Oz)	4.6	0.37	0.079	0.30	+0.00%
`mem_hashtable` (O0)	96.9	5.38	0.056	0.21	+0.00%
`mem_hashtable` (O2)	115.6	11.16	0.097	0.36	-10.89%
`mem_hashtable` (O3)	115.6	11.27	0.098	0.37	-10.89%
`mem_hashtable` (Oz)	135.3	14.60	0.108	0.41	+0.00%
`mem_oob_offset_wrap` (O0)	109.6	2.41	0.022	0.08	+0.00%
`mem_oob_offset_wrap` (O2)	25.5	0.79	0.031	0.12	+0.00%
`mem_oob_offset_wrap` (O3)	36.9	1.15	0.031	0.12	+0.00%
`mem_oob_offset_wrap` (Oz)	61.8	3.07	0.050	0.19	+0.00%
`mem_pointer_chase` (O0)	69.2	9.40	0.136	0.51	+0.00%
`mem_pointer_chase` (O2)	21.8	8.48	0.402	1.57	+0.00%
`mem_pointer_chase` (O3)	21.8	7.25	0.345	1.35	+0.00%
`mem_pointer_chase` (Oz)	35.7	8.95	0.244	0.96	+0.00%
`mem_stream_copy` (O0)	220.3	4.09	0.018	0.07	+0.00%
`mem_stream_copy` (O2)	604.2	14.96	0.025	0.09	-16.66%
`mem_stream_copy` (O3)	327.3	8.03	0.025	0.09	-16.66%
`mem_stream_copy` (Oz)	352.5	17.52	0.050	0.19	+0.00%
`mem_stride_4k` (O0)	318.8	11.50	0.036	0.14	+0.00%
`mem_stride_4k` (O2)	104.9	26.07	0.248	0.93	+0.00%
`mem_stride_4k` (O3)	104.9	25.72	0.245	0.92	+0.00%
`mem_stride_4k` (Oz)	172.0	21.15	0.123	0.46	+0.00%
`mem_stride_64` (O0)	314.6	12.96	0.041	0.16	+0.00%
`mem_stride_64` (O2)	110.1	3.89	0.036	0.13	+0.00%
`mem_stride_64` (O3)	220.2	7.41	0.034	0.13	+0.00%
`mem_stride_64` (Oz)	180.4	6.79	0.038	0.14	+0.00%
`mem_transpose` (O0)	283.1	12.72	0.045	0.17	+0.00%
`mem_transpose` (O2)	43.3	7.22	0.167	0.63	+0.00%
`mem_transpose` (O3)	65.0	10.82	0.166	0.63	+0.00%
`mem_transpose` (Oz)	120.2	12.13	0.100	0.38	+0.00%
`pb_2mm` (O0)	988.3	31.39	0.028	0.10	+0.00%

Continued on next page

Kernel (level)	Fuel (M)	CPU (ms)	μ_ψ (ns)	Cycles/fuel	Winch diff.
pb_2mm (O2)	305.2	11.95	0.038	0.14	+0.00%
pb_2mm (O3)	305.2	15.73	0.052	0.19	+0.00%
pb_2mm (Oz)	388.4	30.69	0.076	0.29	+0.00%
pb_3mm (O0)	1,494.8	25.98	0.018	0.07	+0.00%
pb_3mm (O2)	473.3	16.19	0.034	0.13	+0.00%
pb_3mm (O3)	473.3	16.53	0.041	0.15	+0.00%
pb_3mm (Oz)	577.6	48.96	0.083	0.31	+0.00%
pb_adi (O0)	2,185.4	83.69	0.038	0.14	+0.00%
pb_adi (O2)	637.9	63.75	0.100	0.38	+0.00%
pb_adi (O3)	637.9	63.56	0.100	0.38	+0.00%
pb_adi (Oz)	756.0	50.09	0.066	0.25	+0.00%
pb_atax (O0)	461.6	13.17	0.028	0.11	+0.00%
pb_atax (O2)	276.1	8.37	0.030	0.11	-0.04%
pb_atax (O3)	347.3	10.53	0.030	0.11	-0.04%
pb_atax (Oz)	335.1	11.42	0.034	0.13	+0.00%
pb_bicg (O0)	738.9	13.46	0.018	0.07	+0.00%
pb_bicg (O2)	187.3	9.28	0.050	0.19	-0.04%
pb_bicg (O3)	178.7	6.81	0.038	0.14	-0.04%
pb_bicg (Oz)	189.2	5.16	0.027	0.10	+0.00%
pb_cholesky (O0)	4,891.7	97.55	0.020	0.07	+0.00%
pb_cholesky (O2)	1,564.7	41.22	0.026	0.10	-0.12%
pb_cholesky (O3)	1,564.7	48.10	0.029	0.10	-0.12%
pb_cholesky (Oz)	1,957.9	149.13	0.068	0.25	+0.00%
pb_correlation (O0)	501.8	7.62	0.016	0.06	+0.00%
pb_correlation (O2)	302.2	12.02	0.040	0.15	+0.00%
pb_correlation (O3)	453.3	18.01	0.040	0.15	+0.00%
pb_correlation (Oz)	178.6	11.09	0.062	0.23	+0.00%
pb_covariance (O0)	499.5	7.61	0.016	0.06	+0.00%
pb_covariance (O2)	285.5	16.47	0.058	0.22	+0.00%
pb_covariance (O3)	428.2	24.70	0.058	0.22	+0.00%
pb_covariance (Oz)	177.9	11.10	0.075	0.28	+0.00%
pb_deriche (O0)	194.7	7.46	0.039	0.15	+0.00%
pb_deriche (O2)	180.4	15.05	0.085	0.32	+0.00%
pb_deriche (O3)	240.5	19.88	0.083	0.31	+0.00%
pb_deriche (Oz)	165.5	16.86	0.101	0.38	+0.00%
pb_doitgen (O0)	484.4	8.56	0.021	0.08	+0.00%
pb_doitgen (O2)	277.0	7.31	0.027	0.10	+0.00%
pb_doitgen (O3)	415.5	11.71	0.028	0.10	+0.00%
pb_doitgen (Oz)	590.0	19.52	0.033	0.12	+0.00%
pb_durbin (O0)	830.1	13.70	0.017	0.06	+0.00%
pb_durbin (O2)	508.4	26.02	0.051	0.19	-15.82%
pb_durbin (O3)	403.5	13.04	0.033	0.11	-15.82%
pb_durbin (Oz)	404.6	13.13	0.033	0.12	+0.00%
pb_fdtd_2d (O0)	1,247.3	16.01	0.013	0.05	+0.00%
pb_fdtd_2d (O2)	441.9	8.63	0.020	0.07	+0.00%
pb_fdtd_2d (O3)	441.9	8.62	0.020	0.07	+0.00%
pb_fdtd_2d (Oz)	495.0	32.20	0.064	0.24	+0.00%
pb_floyd_warshall (O0)	11,720.4	186.83	0.016	0.06	+0.00%
pb_floyd_warshall (O2)	3,021.0	138.18	0.046	0.17	+0.00%
pb_floyd_warshall (O3)	3,021.0	52.28	0.017	0.07	+0.00%
pb_floyd_warshall (Oz)	3,650.0	266.59	0.058	0.22	+0.00%
pb_gemm (O0)	727.8	12.53	0.018	0.07	+0.00%
pb_gemm (O2)	677.3	24.72	0.038	0.14	+0.00%
pb_gemm (O3)	677.3	25.05	0.038	0.14	+0.00%
pb_gemm (Oz)	560.4	45.78	0.079	0.30	+0.00%
pb_gemver (O0)	352.5	6.47	0.018	0.07	+0.00%
pb_gemver (O2)	302.3	9.39	0.032	0.12	+0.00%
pb_gemver (O3)	359.8	11.23	0.031	0.12	+0.00%
pb_gemver (Oz)	333.0	9.91	0.030	0.11	+0.00%
pb_gesummv (O0)	262.5	5.86	0.023	0.08	+0.00%
pb_gesummv (O2)	180.4	10.60	0.059	0.22	+0.00%
pb_gesummv (O3)	189.2	11.11	0.058	0.22	+0.00%
pb_gesummv (Oz)	235.4	13.36	0.057	0.21	+0.00%
pb_gramschmidt (O0)	826.2	12.75	0.016	0.06	+0.00%
pb_gramschmidt (O2)	466.7	27.87	0.046	0.17	-0.20%
pb_gramschmidt (O3)	233.4	7.84	0.035	0.11	-0.20%

Continued on next page

Kernel (level)	Fuel (M)	CPU (ms)	μ_ψ (ns)	Cycles/fuel	Winch diff.
pb_gramschmidt (Oz)	274.9	9.82	0.046	0.17	+0.00%
pb_heat_3d (O0)	2,944.4	30.84	0.010	0.04	-0.00%
pb_heat_3d (O2)	709.0	14.92	0.021	0.08	-0.00%
pb_heat_3d (O3)	709.0	14.89	0.021	0.08	-0.00%
pb_heat_3d (Oz)	733.1	14.42	0.020	0.07	-0.00%
pb_jacobi_1d (O0)	369.9	5.36	0.015	0.05	+0.00%
pb_jacobi_1d (O2)	355.6	8.83	0.025	0.09	+0.00%
pb_jacobi_1d (O3)	559.9	14.06	0.028	0.10	+0.00%
pb_jacobi_1d (Oz)	420.8	13.19	0.031	0.12	+0.00%
pb_jacobi_2d (O0)	1,384.0	16.03	0.012	0.04	+0.00%
pb_jacobi_2d (O2)	495.2	10.32	0.021	0.08	+0.00%
pb_jacobi_2d (O3)	495.2	10.30	0.021	0.08	+0.00%
pb_jacobi_2d (Oz)	532.6	28.64	0.052	0.20	+0.00%
pb_lu (O0)	5,592.6	107.95	0.020	0.08	+0.00%
pb_lu (O2)	1,774.3	57.66	0.032	0.12	-0.11%
pb_lu (O3)	1,774.3	54.15	0.032	0.11	-0.11%
pb_lu (Oz)	2,224.9	176.96	0.071	0.27	+0.00%
pb_ludcmp (O0)	5,327.7	126.22	0.023	0.09	+0.00%
pb_ludcmp (O2)	1,713.9	41.58	0.025	0.09	-0.11%
pb_ludcmp (O3)	1,713.6	43.73	0.026	0.09	-0.11%
pb_ludcmp (Oz)	2,186.6	183.82	0.073	0.27	+0.00%
pb_mvt (O0)	278.9	10.14	0.035	0.13	+0.00%
pb_mvt (O2)	285.8	11.61	0.044	0.16	+0.00%
pb_mvt (O3)	339.4	13.81	0.041	0.15	+0.00%
pb_mvt (Oz)	410.9	31.38	0.076	0.29	+0.00%
pb_nussinov (O0)	1,993.3	29.42	0.015	0.06	+0.00%
pb_nussinov (O2)	551.0	11.59	0.021	0.08	-0.18%
pb_nussinov (O3)	551.0	11.56	0.021	0.08	-0.18%
pb_nussinov (Oz)	656.7	41.19	0.061	0.23	+0.00%
pb_seidel_2d (O0)	2,923.3	106.83	0.037	0.14	+0.00%
pb_seidel_2d (O2)	844.0	133.54	0.159	0.60	+0.00%
pb_seidel_2d (O3)	844.0	133.40	0.159	0.60	+0.00%
pb_seidel_2d (Oz)	955.5	133.79	0.140	0.53	+0.00%
pb_symm (O0)	506.7	14.86	0.029	0.11	+0.00%
pb_symm (O2)	545.3	37.57	0.069	0.26	+0.00%
pb_symm (O3)	544.4	12.98	0.024	0.09	+0.00%
pb_symm (Oz)	393.3	25.38	0.065	0.24	+0.00%
pb_syr2k (O0)	593.0	9.17	0.016	0.06	+0.00%
pb_syr2k (O2)	224.1	7.43	0.033	0.13	+0.00%
pb_syr2k (O3)	224.1	7.43	0.033	0.13	+0.00%
pb_syr2k (Oz)	711.3	36.96	0.052	0.20	+0.00%
pb_syrk (O0)	801.3	12.10	0.016	0.06	+0.00%
pb_syrk (O2)	655.3	13.47	0.021	0.08	+0.00%
pb_syrk (O3)	655.3	13.39	0.020	0.08	+0.00%
pb_syrk (Oz)	832.0	59.84	0.072	0.27	+0.00%
pb_trisolv (O0)	309.8	5.27	0.017	0.06	+0.00%
pb_trisolv (O2)	430.5	19.77	0.046	0.17	+0.00%
pb_trisolv (O3)	430.5	22.92	0.051	0.17	+0.00%
pb_trisolv (Oz)	381.9	22.98	0.062	0.23	+0.00%
pb_trmm (O0)	316.6	13.65	0.030	0.11	+0.00%
pb_trmm (O2)	531.6	38.33	0.069	0.26	+0.00%
pb_trmm (O3)	530.6	16.26	0.031	0.12	+0.00%
pb_trmm (Oz)	127.5	3.17	0.025	0.09	+0.00%
quicksort (O0)	184.9	13.63	0.074	0.28	+0.00%
quicksort (O2)	116.7	12.21	0.105	0.39	+0.00%
quicksort (O3)	116.7	12.19	0.105	0.40	+0.00%
quicksort (Oz)	119.7	12.14	0.102	0.38	+0.00%
sha256 (O0)	485.6	26.50	0.051	0.19	+0.00%
sha256 (O2)	444.4	9.69	0.022	0.08	+0.00%
sha256 (O3)	578.8	13.16	0.023	0.09	+0.00%
sha256 (Oz)	530.0	14.25	0.027	0.10	+0.00%
strsearch (O0)	228.6	9.77	0.043	0.16	+0.00%
strsearch (O2)	125.8	9.16	0.073	0.27	+0.00%
strsearch (O3)	209.6	15.21	0.073	0.27	+0.00%
strsearch (Oz)	227.4	12.83	0.057	0.21	+0.00%

Appendix H. Linear-memory probe outcomes

Table 23 gives the outcome counts for each of the six guard configurations and each of the eight probes. Each row covers 100 trials. The campaign produced 3,000 specified returns and 1,800 graceful traps, and no probe ended in a silent corruption or a host crash. The counts in each row sum to the number of trials, so the conformance intervals in Section 5.4 rest on the full set of 4,800 trials.

Table 23: Linear-memory probe outcomes for every configuration and probe. Each probe row covers the 100 trials of one probe under one guard configuration. A specified return and a graceful trap both conform to the specification.

Probe	Trials	Graceful	Spec. return
`default_2gib_guard`			
`grow_one_page`	100	0	100
`grow_past_4gib`	100	0	100
`grow_to_max_minus_one`	100	0	100
`load_addr_4gib_minus_4`	100	100	0
`load_at_zero`	100	0	100
`load_high_half`	100	100	0
`store_last_word_in_bounds`	100	0	100
`store_one_past_end`	100	100	0
`no_growth_reservation`			
`grow_one_page`	100	0	100
`grow_past_4gib`	100	0	100
`grow_to_max_minus_one`	100	0	100
`load_addr_4gib_minus_4`	100	100	0
`load_at_zero`	100	0	100
`load_high_half`	100	100	0
`store_last_word_in_bounds`	100	0	100
`store_one_past_end`	100	100	0
`reduced_64kib_guard`			
`grow_one_page`	100	0	100
`grow_past_4gib`	100	0	100
`grow_to_max_minus_one`	100	0	100
`load_addr_4gib_minus_4`	100	100	0
`load_at_zero`	100	0	100
`load_high_half`	100	100	0
`store_last_word_in_bounds`	100	0	100
`store_one_past_end`	100	100	0
`zero_guard`			
`grow_one_page`	100	0	100
`grow_past_4gib`	100	0	100
`grow_to_max_minus_one`	100	0	100
`load_addr_4gib_minus_4`	100	100	0
`load_at_zero`	100	0	100
`load_high_half`	100	100	0
`store_last_word_in_bounds`	100	0	100
`store_one_past_end`	100	100	0
`zero_guard_explicit_checks`			
`grow_one_page`	100	0	100
`grow_past_4gib`	100	0	100
`grow_to_max_minus_one`	100	0	100
`load_addr_4gib_minus_4`	100	100	0
`load_at_zero`	100	0	100
`load_high_half`	100	100	0
`store_last_word_in_bounds`	100	0	100
`store_one_past_end`	100	100	0
`zero_guard_no_growth_reservation`			
`grow_one_page`	100	0	100

Continued on next page

Payload	Host call	Native	Overhead	Ratio	WASI P1	WASI P2	P2/P1 [95% CI]
64 B	40	19	21	2.10	787	879	1.11 [1.10, 1.12]
256 B	46	25	21	1.84	—	—	—
1 KiB	79	54	24	1.45	943	1,053	1.12 [1.11, 1.12]
4 KiB	175	149	26	1.17	—	—	—
16 KiB	553	511	43	1.08	4,786	3,309	0.69 [0.69, 0.69]
64 KiB	2,359	2,245	114	1.05	—	—	—
256 KiB	8,769	8,727	42	1.00	72,877	45,014	0.62 [0.62, 0.62]
1 MiB	32,410	32,548	-138	1.00	290,389	183,038	0.63 [0.63, 0.63]
4 MiB	133,004	133,051	-47	1.00	1,161,778	735,996	0.63 [0.63, 0.64]
16 MiB	657,883	655,561	2,322	1.00	4,651,030	2,972,038	0.64 [0.64, 0.64]

Probe	Trials	Graceful	Spec. return
`grow_past_4gib`	100	0	100
`grow_to_max_minus_one`	100	0	100
`load_addr_4gib_minus_4`	100	100	0
`load_at_zero`	100	0	100
`load_high_half`	100	100	0
`store_last_word_in_bounds`	100	0	100
`store_one_past_end`	100	100	0

Appendix I. Host-call and WASI sweeps

Table 24 gives the medians behind Figure 7. The host-call side uses the Rust embedder of Section 5.5; overhead is the host-call median minus the native median. The WASI per-write cost is the replicate-wise full-minus-baseline CPU time divided by the number of writes, and the Preview 2 to Preview 1 ratio at each size carries a 95% interval from Welch's t on log costs. The Python-embedder measurements of the earlier version are kept in `data/archive_round3/` for comparison.

Appendix J. Per-kernel Welch ANOVA

Table 25 reports the Welch ANOVA for each of the 50 kernels across the four frontend levels. Every kernel has a significant level effect. The median partial η^2 is 0.981 for the integer kernels, 0.994 for the linear-memory kernels, and 0.968 for PolyBench, with minima of 0.696, 0.775, and 0.664, the pattern summarized in Table 6.

Table 25: Welch's heteroscedastic ANOVA of log-transformed CPU time across the four frontend levels, for each kernel under Wasmtime Cranelift. Degrees of freedom between groups are 3 for every kernel. The within-group degrees of freedom are fractional because of the Welch correction.

Kernel	Group	F_W	df	p	η_p^2
`adv_branch`	integer	71,661.7	212.8	$< 10^{-300}$	0.997
`cachethrash`	integer	101,116.6	213.8	$< 10^{-300}$	0.997
`chacha20`	integer	189,732.4	213.4	$< 10^{-300}$	0.999
`collatz`	integer	7,277.0	213.7	8.00×10^{-215}	0.969
`crc32`	integer	283.9	194.3	9.60×10^{-71}	0.696

Continued on next page

Kernel	Group	F_W	df	p	η_p^2
`divrem`	integer	174,326.0	197.7	$< 10^{-300}$	0.994
`fnv1a`	integer	53,753.6	200.2	1.38×10^{-290}	0.848
`mem_binary_search`	memory	11,473.0	176.3	8.99×10^{-202}	0.935
`mem_fill_reduce`	memory	45,836.2	216.6	2.62×10^{-303}	0.996
`mem_grow_monotonic`	memory	15,155.8	200.2	9.50×10^{-236}	0.960
`mem_hashtable`	memory	34,913.5	203.0	4.66×10^{-275}	0.998
`mem_oob_offset_wrap`	memory	1,347,241.5	207.7	$< 10^{-300}$	1.000
`mem_pointer_chase`	memory	1,109.5	169.0	6.54×10^{-111}	0.775
`mem_stream_copy`	memory	474,967.4	201.2	$< 10^{-300}$	0.998
`mem_stride_4k`	memory	8,356.2	196.9	3.11×10^{-207}	0.990
`mem_stride_64`	memory	55,137.7	191.0	2.94×10^{-280}	0.998
`mem_transpose`	memory	74,609.5	205.5	$< 10^{-300}$	0.993
`pb_2mm`	polybench	3,088.8	167.6	2.11×10^{-146}	0.935
`pb_3mm`	polybench	10,557.7	172.5	3.59×10^{-195}	0.929
`pb_adi`	polybench	83,878.1	218.9	$< 10^{-300}$	0.999
`pb_atax`	polybench	18,360.5	202.6	1.75×10^{-246}	0.963
`pb_bicg`	polybench	194,924.8	202.8	$< 10^{-300}$	0.999
`pb_cholesky`	polybench	3,902.7	166.8	3.34×10^{-154}	0.853
`pb_correlation`	polybench	33,955.4	196.1	4.66×10^{-266}	0.973
`pb_covariance`	polybench	10,337.8	170.7	1.06×10^{-192}	0.880
`pb_deriche`	polybench	12,688.8	179.9	4.24×10^{-209}	0.991
`pb_doitgen`	polybench	8,896.4	170.8	2.46×10^{-187}	0.857
`pb_durbin`	polybench	76,658.4	196.0	1.62×10^{-300}	0.967
`pb_fdtd_2d`	polybench	26,763.1	199.7	7.56×10^{-260}	0.994
`pb_floyd_warshall`	polybench	418,038.7	204.4	$< 10^{-300}$	0.864
`pb_gemm`	polybench	4,490.3	216.5	1.35×10^{-194}	0.969
`pb_gemver`	polybench	13,281.6	201.5	3.33×10^{-231}	0.935
`pb_gesummv`	polybench	16,155.1	186.1	1.22×10^{-224}	0.988
`pb_gramschmidt`	polybench	528.8	192.5	1.21×10^{-92}	0.664
`pb_heat_3d`	polybench	217,146.7	219.1	$< 10^{-300}$	1.000
`pb_jacobi_1d`	polybench	186,920.1	199.3	$< 10^{-300}$	0.976
`pb_jacobi_2d`	polybench	10,922.5	201.3	1.60×10^{-222}	0.980
`pb_lu`	polybench	1,333.3	205.8	1.62×10^{-134}	0.882
`pb_ludcmp`	polybench	6,323.5	209.3	4.93×10^{-205}	0.936
`pb_mvt`	polybench	26,141.4	193.2	9.08×10^{-252}	0.960
`pb_nussinov`	polybench	388,019.1	210.0	$< 10^{-300}$	0.959
`pb_seidel_2d`	polybench	3,173.9	206.6	1.66×10^{-172}	0.980
`pb_symm`	polybench	543,170.1	204.7	$< 10^{-300}$	0.999
`pb_syr2k`	polybench	238,814.8	176.4	$< 10^{-300}$	0.999
`pb_syrk`	polybench	65,288.9	196.6	1.70×10^{-294}	0.998
`pb_trisolv`	polybench	51,027.8	204.5	1.43×10^{-293}	0.991
`pb_trmm`	polybench	441,903.7	196.6	$< 10^{-300}$	0.912
`quicksort`	integer	4,072.8	217.0	1.99×10^{-190}	0.961
`sha256`	integer	15,131.6	204.5	7.95×10^{-240}	0.941
`strsearch`	integer	78,451.4	215.9	$< 10^{-300}$	0.998